EW-RIE*: Làm sạch ma trận tương quan có trọng số mũ bằng lý thuyết ma trận ngẫu nhiên

Ứng dụng: danh mục phương sai nhỏ nhất trên HOSE — **bản v3 trên khung của thầy** (T1 hoàn thiện, dữ liệu HOSE 2015–11/2026 gồm mã hủy niêm yết)

Câu hỏi nghiên cứu. Ở thị trường mới nổi (N cổ phiếu lớn so với số phiên T, biến động thay đổi theo chế độ, cấm bán khống), bộ ước lượng ma trận hiệp phương sai nào cho danh mục rủi ro thấp nhất ngoài mẫu?

Phương pháp đề xuất: EW-RIE*

1. Ước lượng tương quan có trọng số mũ (half-life *h*): $E = \sum_t w_t x_t x_t^\top = C^{1/2} W C^{1/2}$, với $W = \sum_t w_t y_t y_t^\top$ là nhiễu.
2. R-transform của nhiễu: $R_W(g) = \sum_t \dfrac{w_t}{1 - N w_t g}$.
3. Phép thế nhân (BBP 2016): $\xi_i = \lambda_i \dfrac{\operatorname{Im}\gamma(z_i)}{\operatorname{Im} t_E(z_i)}$, $z_i = \lambda_i - i\eta$, $t_E = z g_E(z) - 1$, $\gamma\, R_W(\gamma) = t_E$. Trọng số đều ⇒ trùng RIE cổ điển.
4. Chọn đúng nhánh nghiệm (tiếp tục giải tích), xử lý ngoại lai trên trục thực, hiệu chỉnh N hữu hạn.

Đối chứng: ma trận mẫu, Ledoit–Wolf (2004), NL-LW (2020), MP-clip, RIE (BBP 2016/17), EW-RIE (q_{eff}), EW-NL-LW (**sửa cho N > T_eff**), **EW-WeSpeR (Oriol 2024)**, VS-NL-LW, VS-RIE.

Kết quả chính (§4: HOSE 12/2016–11/2026, 118 kỳ OOS, N=100, T=250, h=60, REB=21, lọc đứng giá ZERO_MAX=20%; Vol ngoài mẫu %/năm):

	Bán khống	**Chỉ mua**
NL-LW	13,10	13,72
EW-NL-LW	12,72	13,12
EW-RIE (q_eff)	14,30	14,76
EW-WeSpeR	12,57	**13,05**
EW-RIE*	**12,56**	13,07
VS-NL-LW	*12,49*	*12,93*

T1 (dữ liệu) — điểm chính

1. Bộ dữ liệu sạch về kỹ thuật (không giá ≤ 0, không khối lượng 0 có giá, không dòng trùng); 5,9% ô thiếu trong khoảng niêm yết; 15,3% phiên đứng giá (r = 0), gắn với thanh khoản kém (ρ hạng = −0,77) ⇒ cần lọc `ZERO_MAX`.
2. 105 mã rời HOSE trong 2015–2026 (85 bắt buộc), **86 mã có giá**, 19 mã thiếu giá.
3. Survivorship bias (T1c): bỏ mã đã hủy làm Vol thấp đi 0,26–0,39 pp và lợi suất cao lên 1,1–2,6 pp/năm; thứ hạng phương pháp không đổi. Chỉ 2 vị thế (trong 118 kỳ × 100 mã) bị hủy ngay trong kỳ nắm giữ.
4. Không lọc đứng giá ⇒ universe có nhiều mã "giả ít rủi ro"; lọc 20% giữ 118/129 kỳ đủ N = 100 mã.

Tài liệu chính: Bun, Bouchaud & Potters (2016) *IEEE Trans. Inf. Theory* 62(12); (2017) *Physics Reports* 666; Ledoit & Wolf (2020) *Ann. Statist.*; Oriol (2024) *WeSpeR*, arXiv:2410.14413; Laloux et al. (1999) *PRL*; Jagannathan & Ma (2003) *J. Finance*.

Phần thay đổi so với file gốc của thầy

Nguyên tắc: các ô của thầy (§1 thư viện ước lượng, §1b kiểm thử, §3 backtest, §4 chạy chính, §5, §6 khung) **giữ nguyên từng chữ** — mọi ô thêm vào đều ở dưới tiêu đề **"▶ v3"** và chỉ *bổ sung* (không sửa ô của thầy). Duy nhất ô **CONFIG** được đổi: thêm `ZERO_MAX`, đổi `DATA_SOURCE` sang `"csv"` và thêm `DATA_DIR`.

#	Việc	Làm ở đâu
1	**Dữ liệu mới** `data_hose/` (vnstock/VCI, 2015-01 → 2026-11, **có mã hủy niêm yết**): `prices.csv`, `values.csv` đọc bằng hàm của thầy; tự tải từ GitHub nếu chạy trên web	§2, ▶ T1.0
2	**T1 hoàn thiện:** chỉ giữ quan sát trong khoảng niêm yết HOSE (`security_master.csv`); lịch sử niêm yết / hủy niêm yết (T1a); kiểm tra chất lượng + trực quan hóa + đánh giá tổng quan (T1b); đo survivorship bias (T1c)	§2, §4
3	**Lọc mã đứng giá** `ZERO_MAX = 0,20` (tỷ lệ phiên r = 0 trong cửa sổ quá khứ), đặt ở CONFIG; áp dụng qua bản `run_backtest` sinh từ chính mã nguồn của thầy	§3
4	**Sửa EW-NL-LW khi N > T_eff** (nhánh p > n của Ledoit–Wolf 2020; trùng hệt bản thầy khi N ≤ T_eff)	▶ §1c, kiểm thử §1b
5	**Đối chứng EW-WeSpeR** (Oriol 2024, arXiv:2410.14413)	▶ §1c, kiểm thử §1b, §4
6	Mã hủy niêm yết: lợi suất **0 sau ngày giao dịch cuối** (đúng quy ước của thầy "mã ngừng GD ngoài mẫu → lợi suất 0")	§3, T1c

T2, T3, T5, T6 giữ khung trống như file của thầy — chỉ tinh chỉnh siêu tham số sau khi T1 được duyệt.

Phân công nhóm

Mã	Việc	Đầu ra	Mục trong notebook
T1 ✔	Dữ liệu 2020–2025 (vnstock/nguồn trả phí) + **mã đã hủy niêm yết** để loại survivorship bias — *đã làm: HOSE 2015–11/2026, 512 mã gồm 105 mã rời sàn*	`prices.csv`, `values.csv`	§2 (T1a, T1b), §4 (T1c)
T2	Độ vững: chi phí GD thực tế (phí + thuế 0,1%), tần suất tái cân bằng 5/21/63 phiên, xử lý phiên không giao dịch (~18%)	Bảng độ vững	§6, ô T2
T3	Đối chứng & kiểm định chuẩn: danh mục 1/N, mô hình nhân tố, DCC-GARCH; kiểm định chênh lệch phương sai Ledoit–Wolf (2011, HAC), Sharpe (Jobson–Korkie/Memmel)	Bảng kiểm định	§6, ô T3
T4	Lý thuyết: chứng minh tính đúng của việc chọn nhánh, điều kiện ngoại lai, hiệu chỉnh N hữu hạn; **rà soát tính mới** (đọc toàn văn BBP 2016 §III và Phys. Rep. 2017)	Mục Methodology (LaTeX)	—
T5	Vì sao VS giúp trên HOSE nhưng không trong GARCH? (biến động riêng lẻ, tương quan đổi theo chế độ...)	Ghi chú + thí nghiệm	§6, ô T5
T6	Mở rộng: N = 50/150, cửa sổ T, VN30/HNX	Bảng phụ lục	§5 (đổi CONFIG)
T7	Viết bài, nhắm tạp chí (kiểm tra hạng Q trên Scimago)	Bản thảo	—

Quy ước: mỗi người làm trên bản sao notebook (`File → Save a copy in Drive`), ghi kết quả vào thư mục chung. Đổi tham số ở ô CONFIG, không sửa thư viện ở §1 trừ khi đã thống nhất.

0. Cài đặt & cấu hình

In [1]:
```
import os, sys, subprocess, time, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf
warnings.filterwarnings("ignore")

IN_COLAB = "google.colab" in sys.modules
print("Colab:", IN_COLAB, "| numpy", np.__version__, "| pandas",
pd.__version__)
```

```
Colab: False | numpy 2.5.3 | pandas 3.0.5
```

In [2]:
```
# ======================== CONFIG ========================
CONFIG = dict(
    N=100,            # số mã thanh khoản nhất mỗi kỳ
    WINDOW=250,       # cửa sổ ước lượng (phiên)
    REB=21,           # tái cân bằng mỗi REB phiên
    COST=0.0025,      # chi phí 1 chiều (giả định, T2 cần chỉnh)
    HL_EW=60,         # half-life trọng số mũ
    HL_VOL=20,        # half-life chuẩn hóa biến động (VS)
    MAX_ABS_RET=0.15, # |r| > ngưỡng coi là lỗi dữ liệu (biên độ HOSE ±7%)
    ZERO_MAX=0.20,    # v3: loại mã đứng giá — tỷ lệ phiên r = 0 trong cửa sổ quá khứ > ZERO_MAX (1.0 = không lọc, như bản thảo)
    QUICK=False,      # True: chạy nhanh với ít bộ ước lượng
)
DATA_SOURCE = "csv"      # v3: "csv" = data_hose/ (2015–11/2026, có mã huỷ niêm yết) | "github" (2006–3/2020, cũ) | "vnstock"
# v3: thư mục data_hose/ có sẵn cạnh notebook thì dùng; không có (chạy trên web/Colab) thì ô T1.0 tự tải từ GitHub
DATA_DIR = next((os.path.abspath(d) for d in ("../data_hose", "./data_hose", "data_hose") if os.path.exists(f"{d}/prices.csv")), None)
WORKDIR = os.environ.get("RMT_WORKDIR", "/content/rmt" if IN_COLAB else "./rmt_work")
os.makedirs(WORKDIR, exist_ok=True)
CONFIG
```

Out[2]:
```
{'N': 100,
 'WINDOW': 250,
 'REB': 21,
 'COST': 0.0025,
 'HL_EW': 60,
 'HL_VOL': 20,
 'MAX_ABS_RET': 0.15,
 'ZERO_MAX': 0.2,
 'QUICK': False}
```

1. Thư viện: các bộ ước lượng

Không cần sửa. Mỗi bộ ước lượng nhận ma trận lợi suất `X` (T×N) và trả về ma trận hiệp phương sai N×N.

In [3]:

```
# ---------- tiện ích ----------
def cov_to_corr(S):
    sd = np.sqrt(np.diag(S)); return S / np.outer(sd, sd), sd

def renorm(C):
    d = np.sqrt(np.diag(C)); return C / np.outer(d, d)

# ---------- đối chứng cổ điển ----------
def est_sample(X): return np.cov(X, rowvar=False)

def est_lw(X): return LedoitWolf().fit(X).covariance_

def nl_lw_S(S, n):
    # Ledoit & Wolf (2020), co phi tuyến giải tích; S: hiệp phương sai, n: cỡ mẫu (hiệu dụng)
    p = len(S); lam, U = np.linalg.eigh(S); lam = np.maximum(lam, 1e-12)
    L = np.tile(lam, (p, 1)).T; H = n ** (-1 / 3) * L.T; x = (L - L.T) / H
    ft = (3 / 4 / np.sqrt(5)) * np.mean(np.maximum(1 - x**2 / 5, 0) / H, axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        Hf = (-3/10/np.pi) * x + (3/4/np.sqrt(5)/np.pi) * (1 - x**2/5) * np.log(np.abs((np.sqrt(5) - x) / (np.sqrt(5) + x)))
    e = np.isclose(np.abs(x), np.sqrt(5)); Hf[e] = (-3/10/np.pi) * x[e]
    Ht = np.mean(Hf / H, axis=1); c = p / n
    d = lam / ((np.pi*c*lam*ft)**2 + (1 - c - np.pi*c*lam*Ht)**2)
    return (U * d) @ U.T

def est_nllw(X):
    Xc = X - X.mean(0); n = len(X) - 1
    return nl_lw_S(Xc.T @ Xc / n, n)

def mp_corr(C, q):
    lam, V = np.linalg.eigh(C); N = len(C)
    lp = (1 - lam[-1] / N) * (1 + np.sqrt(q))**2
    noise = lam < lp; lam = lam.copy()
    if noise.any(): lam[noise] = lam[noise].mean()
    return renorm((V * lam) @ V.T)

def rie_corr(C, q):
    # RIE Bun–Bouchaud–Potters (2016) + hiệu chỉnh N hữu hạn (2017, §8). Đây là bản GỐC (đối chứng).
    N = len(C); lam, V = np.linalg.eigh(C); lam = np.maximum(lam, 1e-10)
    z = lam - 1j / np.sqrt(N); g = (1 / (z[:, None] - lam[None, :])).mean(1)
    xi = lam / np.abs(1 - q + q*z*g)**2
    s2 = lam[0] / (1 - np.sqrt(q))**2 if q < 1 else lam.mean()
    lp, lm = s2 * (1 + np.sqrt(q))**2, s2 * (1 - np.sqrt(q))**2
    gmp = (z + s2*(q - 1) - np.sqrt(z - lm) * np.sqrt(z - lp)) / (2*q*z*s2)
    xi = xi * np.maximum(1, s2 * np.abs(1 - q + q*z*gmp)**2 / lam)
    xi *= N / xi.sum(); return renorm((V * xi) @ V.T)

def _corr_wrap(cleaner):
    def est(X):
        T, N = X.shape; C, sd = cov_to_corr(np.cov(X, rowvar=False))
        return cleaner(C, N / T) * np.outer(sd, sd)
    return est
est_mp, est_rie = _corr_wrap(mp_corr), _corr_wrap(rie_corr)
```

In [4]:

```python
# ---------- EW (trọng số mũ) và VS (chuẩn hóa biến động) ----------
def ew_weights(T, hl):
    w = 0.5 ** ((T - 1 - np.arange(T)) / hl); return w / w.sum()

def ew_corr(X, w):
    Xc = X - w @ X; S = (Xc * w[:, None]).T @ Xc
    sd = np.sqrt(np.diag(S)); return S / np.outer(sd, sd), sd

def make_ew(cleaner, hl):
    # cleaner(C, w) -> tương quan đã làm sạch
    def est(X):
        w = ew_weights(len(X), hl); C, sd = ew_corr(X, w)
        return cleaner(C, w) * np.outer(sd, sd)
    return est

def ewma_standardize(X, hl):
    lam = 0.5 ** (1 / hl); s2 = X[:hl].var(0) + 1e-10; Z = np.empty_like(X)
    for t in range(len(X)):
        Z[t] = X[t] / np.sqrt(s2); s2 = lam * s2 + (1 - lam) * X[t]**2
    return Z, np.sqrt(s2)

def make_vs(corr_cleaner, hl_vol):
    # corr_cleaner(Z) -> ma trận tương quan đã làm sạch của lợi suất chuẩn hóa Z
    def est(X):
        Z, sig = ewma_standardize(X, hl_vol); Z = Z[hl_vol:]
        return corr_cleaner(Z) * np.outer(sig, sig)
    return est
```

In [5]:

```
# ---------- EW-RIE*: RIE chính xác cho trọng sô´ mũ (phương pháp đê` xuâ´t)
----------
_STATS = {"fail": 0, "total": 0}; _NULL_CACHE = {}

def _F(g, t, a, w):
    d = 1 - a[None, :] * g[:, None]; R = (w[None, :] / d).sum(1)
    return g * R - t, R, (w[None, :] * a[None, :] / d**2).sum(1)

def _solve_gamma(t, a, w, g0, iters=30):
    # Newton có tìm kiê´m lùi cho  γ·R_W(γ) = t  (vector phức), a = N·w
    g = g0.copy(); f, R, dR = _F(g, t, a, w)
    for _ in range(iters):
        step = f / (R + g * dR); lam_ = np.ones(len(g))
        for _ in range(30):
            gn = g - lam_ * step; fn, Rn, dRn = _F(gn, t, a, w)
            bad = ~(np.abs(fn) < np.abs(f)) & (np.abs(f) > 1e-14)
            if not bad.any(): break
            lam_[bad] *= 0.5
        keep = bad | ~np.isfinite(fn)
        gn[keep], fn[keep], Rn[keep], dRn[keep] = g[keep], f[keep], R[keep],
dR[keep]
        g, f, R, dR = gn, fn, Rn, dRn
        if np.max(np.abs(f)) < 1e-13: break
    return g, np.abs(f)

def _continuation(t, a, w, steps=60, strict=True):
    # Nhánh chính: γ ≈ t khi t→0, đi theo tia s·t (s: 0→1)
    g = 1e-3 * t
    for s in np.geomspace(1e-3, 1.0, steps):
        g, res = _solve_gamma(s * t, a, w, g)
    ok = np.isfinite(g) & (res < 1e-8)
    if strict and not ok.all(): raise RuntimeError("Newton không hội tụ")
    return g if strict else (g, ok)

def _xi(spec, x, w, N, eta):
    z = x - 1j * eta; g = (1 / (z[:, None] - spec[None, :])).mean(1); t = z *
g - 1
    gam, ok = _continuation(t, N * w, w, strict=False)
    gam[~ok] = (t / (1 + N * (w**2).sum() * t))[~ok]       # dự phòng (thực
tê´: 0 lâ`n trên HOSE)
    _STATS["fail"] += int((~ok).sum()); _STATS["total"] += len(ok)
    return x * gam.imag / t.imag

def _xi_outliers(lam, idx, w, N):
    a = N * w; out = np.empty(len(idx))
    for k, i in enumerate(idx):
        g = np.sum(1 / (lam[i] - np.delete(lam, i))) / N     # Stieltjes bo'-
một trên trục thực
        t = np.array([lam[i] * g - 1 + 0j])
        try:
            gam = _continuation(t, a, w); _, R, dR = _F(gam, t, a, w)
            out[k] = lam[i] * np.real(1 / (R + gam * dR))[0]   # ξ = λ·γ'(t)
        except RuntimeError:
            out[k] = _xi(lam, lam[i:i+1], w, N, 1 / np.sqrt(N))[0]
    return out

def _null_spectrum(N, w, reps=8, seed=0):
    key = (N, len(w), round(float(w[-1]), 12))
    if key not in _NULL_CACHE:
        rng = np.random.default_rng(seed)
        ev = [np.linalg.eigvalsh((Y * w[:, None]).T @ Y) for Y in
(rng.standard_normal((len(w), N)) for _ in range(reps))]
        _NULL_CACHE[key] = np.sort(np.concatenate(ev))
    return _NULL_CACHE[key]

def ew_rie_corr(C, w, debias=True):
    N = len(C); lam, V = np.linalg.eigh(C); lam = np.maximum(lam, 1e-10); eta
= 1 / np.sqrt(N)
    null = _null_spectrum(N, w); s2 = np.median(lam) / np.median(null)
    D = lam[:, None] - lam[None, :]
    dens = (eta / (D**2 + eta**2)).sum(1) - 1 / eta            # mật độ lân
cận (bo' tự thân)
```

```
    out = np.where(dens < 0.1 / eta)[0]; bulk = np.setdiff1d(np.arange(N),
out)
    xi = np.empty(N)
    xi[bulk] = _xi(lam, lam[bulk], w, N, eta)
    if len(out): xi[out] = _xi_outliers(lam, out, w, N)
    if debias and len(bulk):
        xi_null = _xi(null, lam[bulk] / s2, w, N, eta)
        xi[bulk] *= np.maximum(1, 1 / np.maximum(xi_null, 1e-6))
    xi = np.maximum(xi, 1e-8); xi *= N / xi.sum()
    return renorm((V * xi) @ V.T)
```

In [6]:

```
# ---------- Danh sách bộ ước lượng ----------
def build_estimators(cfg):
    hl, hv = cfg["HL_EW"], cfg["HL_VOL"]
    E = {
        "Sample": est_sample,
        "Ledoit-Wolf": est_lw,
        "NL-LW": est_nllw,
        "MP-clip": est_mp,
        "RIE": est_rie,
        "VS-NL-LW": make_vs(lambda Z: renorm(nl_lw_S(np.cov(Z, rowvar=False),
len(Z) - 1)), hv),
        "VS-RIE": make_vs(lambda Z: rie_corr(np.corrcoef(Z, rowvar=False),
Z.shape[1] / len(Z)), hv),
        "EW-NL-LW": make_ew(lambda C, w: renorm(nl_lw_S(C, 1 /
(w**2).sum())), hl),
        "EW-RIE (q_eff)": make_ew(lambda C, w: rie_corr(C, len(C) *
(w**2).sum()), hl),
        "EW-RIE*": make_ew(ew_rie_corr, hl),
    }
    if cfg["QUICK"]:
        E = {k: E[k] for k in ["Sample", "NL-LW", "RIE", "EW-NL-LW", "EW-
RIE*"]}
    return E
```

▶ v3 · §1c. Bổ sung (ô mới — không sửa các ô §1 của thầy)

- `nl_lw_S` được **định nghĩa lại** để đúng khi N > T_eff (bản của thầy giữ lại làm `nl_lw_S_v2` để đối chiếu). Các bộ ước lượng của thầy gọi `nl_lw_S` theo tên nên tự dùng bản mới.
- **EW-WeSpeR:** đối chứng kiểu QuEST cho hiệp phương sai có trọng số (Oriol 2024). Khôi phục phổ quần thể τ sao cho phân vị phổ mẫu tiệm cận khớp trị riêng quan sát, rồi co phi tuyến bằng công thức oracle d(λ) = Im e(λ)/Im m(λ). Cài lại bằng numpy từ các phương trình của bài (không phải mã PyTorch gốc của tác giả).
- `build_estimators` được bọc để thêm "EW-WeSpeR" vào danh sách; ô §4 của thầy chạy nguyên văn.

In [7]:

```
import hashlib
nl_lw_S_v2 = nl_lw_S        # bản của thầy (chỉ đúng khi N ≤ T_eff), giữ để đối chiếu

# ---------- v3: NL-LW cho cả trường hợp p > n (N > T_eff) ----------
def nl_lw_S(S, n):
    # Ledoit & Wolf (2020), co phi tuyến giải tích; S: hiệp phương sai, n: cỡ mẫu (hiệu dụng, có thể không nguyên).
    # p <= n: công thức gốc (giống hệt v2).  p > n: nhánh "p > n" trong mã gốc analytical_shrinkage.m của LW (2020):
    # chỉ [n] trị riêng lớn nhất được xem là "khác 0" (dùng cho mật độ nhân), các trị riêng còn lại nhận chung d̃₀
    # (d̃₀ dựa trên Hilbert transform tại 0). v2 dùng công thức p <= n với c = p/n > 1 ⇒ mẫu số (1 − c − πcλH)² sai, co sai các trị riêng nhỏ.
    p = len(S); lam, U = np.linalg.eigh(S); lam = np.maximum(lam, 1e-12)
    m = p if p <= n else max(int(np.floor(n)), 2)
    lz = lam[p - m:]; h = n ** (-1 / 3)
    L = np.tile(lz, (m, 1)).T; H = h * L.T; x = (L - L.T) / H
    ft = (3 / 4 / np.sqrt(5)) * np.mean(np.maximum(1 - x**2 / 5, 0) / H, axis=1)
    with np.errstate(divide="ignore", invalid="ignore"):
        Hf = (-3/10/np.pi) * x + (3/4/np.sqrt(5)/np.pi) * (1 - x**2/5) * np.log(np.abs((np.sqrt(5) - x) / (np.sqrt(5) + x)))
    e = np.isclose(np.abs(x), np.sqrt(5)); Hf[e] = (-3/10/np.pi) * x[e]
    Ht = np.mean(Hf / H, axis=1); c = p / n
    if p <= n:
        d = lz / ((np.pi*c*lz*ft)**2 + (1 - c - np.pi*c*lz*Ht)**2)
    else:
        Hf0 = (1/np.pi) * (3/10/h**2 + 3/4/np.sqrt(5)/h * (1 - 1/5/h**2) * np.log((1 + np.sqrt(5)*h) / (1 - np.sqrt(5)*h))) * np.mean(1 / lz)
        d0 = 1 / (np.pi * (p - n) / n * Hf0)
        d1 = lz / (np.pi**2 * lz**2 * (ft**2 + Ht**2))
        d = np.concatenate([np.full(p - m, d0), d1])
    return (U * d) @ U.T
```

In [8]:
```
# ---------- v3: EW-WeSpeR — đổi chứng kiểu QuEST cho hiệp phương sai có trọng số (Oriol 2024, arXiv:2410.14413) ----------
# Mô hình: E = Σ_t w_t x_t x_tᵀ, x_t = C^{1/2} y_t, Σw = 1. Với r(e) = Σ_t w_t / (1 + N w_t e) (= R_W(−e)):
#   e(z) = (1/N) Σ_i τ_i / (τ_i r(e(z)) − z),      m(z) = (1/N) Σ_i 1 / (τ_i r(e(z)) − z)     (Im z > 0)
# m là Stieltjes của phổ mẫu tiệm cận (mật độ ρ = Im m / π); e = (1/N) tr C·G(z). Cơ phi tuyến "oracle":
#   d(λ) = lim u_λᵀ C u_λ = Im e(λ + i0) / Im m(λ + i0)      (trọng số đều ⇒ λ / |1 − q − qλ m|², Ledoit–Péché).
# WeSpeR = (1) tìm phổ quần thể τ sao cho phân vị của ρ_τ khớp trị riêng mẫu (như QuEST, nhưng với trọng số),
#          (2) áp d(λ_i) với τ đã khớp. Khác EW-RIE*: EW-RIE* dùng trực tiếp Stieltjes thực nghiệm (làm trơn η = N^{-1/2}),
#          WeSpeR dùng Stieltjes của MÔ HÌNH đã khớp ⇒ không cần η, không cần xử lý ngoại lai/hiệu chỉnh N hữu hạn riêng.
# Ghi chú: cài lại bằng numpy từ các phương trình trên (bản gốc của tác giả viết bằng PyTorch + autodiff; phiên làm việc này
# không truy cập được arXiv). τ tham số hóa: log τ tuyến tính từng khúc theo hạng (K nút) + các ngoại lai tự do;
# tối ưu bằng least_squares (Levenberg–Marquardt kiểu trf), sai số tương đối log λ̂_i − log λ_i; khởi tạo ấm từ kỳ trước.
_WSP = {"nfev": [], "cost": [], "warm": {}}

def _wsp_bins(w, nb=40):
    # nén phân phối trọng số thành nb nhóm (r chỉ phụ thuộc phân phối của w): giá trị TB và khối lượng Σw mỗi nhóm
    ws = np.sort(w); g = np.array_split(ws, min(nb, len(ws)))
    return np.array([x.mean() for x in g]), np.array([x.sum() for x in g])

def _wsp_solve(z, tau, wv, wm, N, e0=None, iters=40):
    # Newton (có tìm kiếm lùi, giữ Im e > 0) cho F(e) = e − mean τ/(τ r(e) − z), vector theo z
    def F(e):
        den = 1 + N * wv[None, :] * e[:, None]
        r = (wm[None, :] / den).sum(1); dr = -(N * wm[None, :] * wv[None, :] / den**2).sum(1)
        q = tau[None, :] * r[:, None] - z[:, None]
        f = e - (tau[None, :] / q).mean(1)
        df = 1 + (tau[None, :]**2 * dr[:, None] / q**2).mean(1)
        return f, df, q
    e = (tau[None, :] / (tau[None, :] - z[:, None])).mean(1) if e0 is None else e0.copy()
    f, df, q = F(e)
    for _ in range(iters):
        st = f / df; a = np.ones(len(e))
        for _ in range(25):
            en = e - a * st; fn, dfn, qn = F(en)
            bad = (~np.isfinite(fn)) | (np.abs(fn) > np.abs(f)) | (en.imag <= 0)
            bad &= np.abs(f) > 1e-13
            if not bad.any(): break
            a[bad] *= 0.5
        keep = bad
        en[keep], fn[keep], dfn[keep], qn[keep] = e[keep], f[keep], df[keep], q[keep]
        e, f, df, q = en, fn, dfn, qn
        if np.max(np.abs(f)) < 1e-11: break
    return e, (1 / q).mean(1), np.abs(f)

def _wsp_cold(x, eta, tau, wv, wm, N, steps=14):
    # tiếp tục theo η: từ η lớn (e ≈ mean τ/(τ − z)) xuống η đích ⇒ đúng nhánh Stieltjes (Im e > 0)
    e = None
    for s in np.geomspace(10 * max(x.max(), tau.max()), eta.min(), steps):
        e, m, res = _wsp_solve(x + 1j * np.maximum(eta, s), tau, wv, wm, N, e)
    return _wsp_solve(x + 1j * eta, tau, wv, wm, N, e)

def _wsp_quantiles(x, m, n_nz, N):
    # phân vị của phần khối lượng dương của ρ = Im m/π trên lưới x (hình thang), khớp với n_nz trị riêng khác 0
```

```
    rho = np.maximum(m.imag, 0) / np.pi; cdf = np.concatenate([[0],
np.cumsum(0.5 * (rho[1:] + rho[:-1]) * np.diff(x))])
    cdf /= cdf[-1]; u = (np.arange(n_nz) + 0.5) / n_nz
    return np.interp(u, cdf, x)

def _wsp_fwd(x, eta, tau, wv, wm, N, e0=None):
    # giải e trên lưới (ấm nếu có e0, nếu không/không hội tụ thì tiếp
tục theo η) + các đạo hàm theo τ (đạo hàm ẩn)
    ok = False
    if e0 is not None:
        e, m, res = _wsp_solve(x + 1j * eta, tau, wv, wm, N, e0, iters=15);
ok = np.isfinite(e).all() and res.max() < 1e-9
    if not ok: e, m, res = _wsp_cold(x, eta, tau, wv, wm, N)
    return e, m, res

def _wsp_jac_m(x, eta, e, tau, wv, wm, N):
    # dm/dτ_j (G×N):  F(e,τ) = e − mean τ/q, q = τ r(e) − z
    z = x + 1j * eta; den = 1 + N * wv[None, :] * e[:, None]
    r = (wm[None, :] / den).sum(1); dr = -(N * wm[None, :] * wv[None, :] /
den**2).sum(1)
    q = tau[None, :] * r[:, None] - z[:, None]; q2 = q**2
    dF = 1 + (tau[None, :]**2 * dr[:, None] / q2).mean(1)
    de = -(z[:, None] / (N * q2)) / dF[:, None]
    A = -(tau[None, :] * dr[:, None] / q2).mean(1)
    return -(r[:, None] / (N * q2)) + A[:, None] * de

def ew_wesper_corr(C, w, K=8, max_nfev=25, G=350, return_info=False):
    from scipy.optimize import least_squares
    N = len(C); lam, V = np.linalg.eigh(C); T = len(w)
    n_nz = min(N, T - 1); lz = np.maximum(lam[N - n_nz:], 1e-10)          #
trị riêng khác 0 (tăng dần)
    wv, wm = _wsp_bins(w)
    eta0 = 1 / np.sqrt(N); D = lz[:, None] - lz[None, :]                    #
ngoại lai: như ew_rie_corr
    dens = (eta0 / (D**2 + eta0**2)).sum(1) - 1 / eta0
    n_out = int(min(np.sum(dens < 0.1 / eta0), 5)); nb = N - n_out
    knots = np.linspace(0, 1, K); rk = (np.arange(nb) + 0.5) / nb
    def tau_of(th):
        tb = np.exp(np.interp(rk, knots, np.cumsum(np.concatenate([[th[0]],
np.exp(th[1:K])]))))
        return np.concatenate([tb, np.exp(th[K:])]) if n_out else tb
    x = np.unique(np.concatenate([np.geomspace(lz[0] * .25, lz[-1] * 1.6, G),
                                  *[lz[-k] * np.linspace(.6, 1.4, 60) for k
in range(1, n_out + 1)]]))
    eta = 2e-3 * x; dx = np.diff(x); u = (np.arange(n_nz) + 0.5) / n_nz
    key = (N, T, K, n_out, round(float(w[-1]), 12))
    qb = np.maximum.accumulate(np.log(np.quantile(lz[:n_nz - n_out] if n_nz >
n_out else lz, knots)))
    th_cold = np.concatenate([[qb[0]], np.log(np.maximum(np.diff(qb), 1e-3)),
np.log(lz[n_nz - n_out:] * .9) if n_out else []])
    st = {}
    def evaluate(th):
        if st.get("th") is not None and np.array_equal(st["th"], th): return
        tau = tau_of(th); e, m, res = _wsp_fwd(x, eta, tau, wv, wm, N,
st.get("e"))
        rho = np.maximum(m.imag, 0) / np.pi; raw = np.concatenate([[0],
np.cumsum(0.5 * (rho[1:] + rho[:-1]) * dx)])
        tot = raw[-1]; cdf = raw / tot; qh = np.interp(u, cdf, x)
        st.update(th=th.copy(), tau=tau, e=e, m=m, rho=rho, raw=raw, tot=tot,
cdf=cdf, qh=qh, ok=np.isfinite(qh).all())
    def fun(th):
        evaluate(th); return np.log(st["qh"]) - np.log(lz) if st["ok"] else
np.full(n_nz, 10.0)
    def jac(th):
        evaluate(th)
        dm = _wsp_jac_m(x, eta, st["e"], st["tau"], wv, wm, N); dr = dm.imag
/ np.pi
        draw = np.vstack([np.zeros(N), np.cumsum(0.5 * (dr[1:] + dr[:-1]) *
dx[:, None], axis=0)])
        dcdf = (draw - st["cdf"][:, None] * draw[-1][None, :]) / st["tot"]
        k = np.clip(np.searchsorted(x, st["qh"]) - 1, 0, len(x) - 2); f =
(st["qh"] - x[k]) / dx[k]
        dC = dcdf[k] * (1 - f)[:, None] + dcdf[k + 1] * f[:, None]
```

```
        pdf = np.maximum((st["rho"][k] * (1 - f) + st["rho"][k + 1] * f) /
st["tot"], 1e-3 / (lz[-1]))
        Jt = -dC / pdf[:, None] / st["qh"][:, None]
# ∂ log λˆ / ∂τ
        h = 1e-6; t0 = tau_of(th)
        Tt = np.column_stack([(tau_of(th + h * np.eye(len(th))[j]) - t0) / h
for j in range(len(th))])
        return Jt @ Tt
    fit = lambda th0: (st.clear(), least_squares(fun, th0, jac=jac,
method="trf", max_nfev=max_nfev, xtol=1e-4, ftol=1e-4))[1]
    sol = fit(_WSP["warm"].get(key, th_cold))                # khởi tạo â´m
từ kỳ trước (cùng N, T, h) nê´u có
    if key in _WSP["warm"] and np.sqrt(np.mean(sol.fun**2)) > 0.05:  # â´m
hởng (cực tiểu địa phương) ⇒ khớp lại từ đầ̀u, giữ nghiệm tô´t hơn
        s2 = fit(th_cold); sol = s2 if np.sum(s2.fun**2) < np.sum(sol.fun**2)
else sol
    _WSP["warm"][key] = sol.x; _WSP["nfev"].append(sol.nfev);
_WSP["cost"].append(np.sqrt(np.mean(sol.fun**2)))
    tau = tau_of(sol.x)
    # co: d(λ_i) = Im e / Im m tại λ_i + iη (η nhở; ở khe/ngoài giá đỡ tỷ̉
sô´ → e'(λ)/m'(λ), vâ˜n xác định)
    ee, mm, _ = _wsp_cold(lz, 1e-4 * lz, tau, wv, wm, N); d = ee.imag /
mm.imag
    xi = np.empty(N); xi[N - n_nz:] = np.maximum(d, 1e-8)
    if n_nz < N:   # không gian null (N ≥ T): giữ vê´t (tr C = N) cho các
hướng null
        xi[:N - n_nz] = max((N - xi[N - n_nz:].sum()) / (N - n_nz), 1e-3 *
np.mean(xi[N - n_nz:]))
    xi *= N / xi.sum(); out = renorm((V * xi) @ V.T)
    return (out, dict(tau=tau, xi=xi, lam=lam, rmse=_WSP["cost"][-1],
nfev=sol.nfev, n_out=n_out)) if return_info else out


# Bọc an toàn + cache theo (C, w): cùng cửa sổ ở hai chê´ độ chỉ̉ ước
lượng 1 lần; lô˜i sô´ ⇒ dự phòng EW-RIE* (được đê´m)
_WSP_SAFE = {"fallback": 0, "total": 0}; _WSP_CACHE = {}
def ew_wesper_safe(C, w):
    key = (hashlib.md5(C.tobytes()).hexdigest(), len(w), round(float(w[-1]),
14))
    if key not in _WSP_CACHE:
        _WSP_SAFE["total"] += 1
        try:
            out = ew_wesper_corr(C, w); assert np.isfinite(out).all()
        except Exception:
            _WSP_SAFE["fallback"] += 1; out = ew_rie_corr(C, w)
        _WSP_CACHE[key] = out
    return _WSP_CACHE[key]

_build_estimators_thay = build_estimators           # danh sách cửa thầy
def build_estimators(cfg):
    E, out = _build_estimators_thay(cfg), {}
    for k, f in E.items():
        if k == "EW-RIE*": out["EW-WeSpeR"] = make_ew(ew_wesper_safe,
cfg["HL_EW"])
        out[k] = f
    return out
```

1b. Kiểm thử (chạy trước mọi thay đổi)

1. Trọng số đều ⇒ EW-RIE* trùng RIE cổ điển. 2) So với oracle trên dữ liệu tổng hợp đã biết C thật.

In [9]:
```
rng = np.random.default_rng(0); N_, T_ = 100, 250
X = rng.standard_normal((T_, N_)); Cs = np.corrcoef(X, rowvar=False); lam =
np.linalg.eigvalsh(Cs)
z = lam - 0.1j; g = (1 / (z[:, None] - lam[None, :])).mean(1); q = N_ / T_
err = np.max(np.abs(_xi(lam, lam, np.full(T_, 1 / T_), N_, 0.1) - lam /
np.abs(1 - q + q*z*g)**2) / lam)
print(f"[Test 1] trọng số đều: sai lệch tương đối max = {err:.1e}"); assert
err < 1e-10
```

```
[Test 1] trọng số đều: sai lệch tương đối max = 2.9e-15
```

In [10]:
```
def gmv(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def factor_corr(N, rng):
    sec = rng.integers(0, 8, N); B = np.zeros((N, 9))
    B[:, 0] = rng.uniform(.4, .8, N); B[np.arange(N), 1 + sec] =
rng.uniform(.2, .5, N)
    return renorm(B @ B.T + np.diag(rng.uniform(.3, .6, N)))

rows = []
for hl in (30, 60):
    w = ew_weights(T_, hl); acc = {}
    for rep in range(6):
        Ct = factor_corr(N_, rng); X = rng.standard_normal((T_, N_)) @
np.linalg.cholesky(Ct).T
        C, _ = ew_corr(X, w); lam, V = np.linalg.eigh(C)
        Co = renorm((V * np.einsum("ij,jk,ki->i", V.T, Ct, V)) @ V.T)    #
oracle RIE
        wo = gmv(Ct); best = wo @ Ct @ wo
        for k, Ch in {"EW-Sample": C, "EW-RIE (q_eff)": rie_corr(C, N_ *
(w**2).sum()),
                      "EW-NL-LW": renorm(nl_lw_S(C, 1 / (w**2).sum())), "EW-
RIE*": ew_rie_corr(C, w),
                      "Oracle": Co}.items():
            wh = gmv(Ch); acc.setdefault(k, []).append(wh @ Ct @ wh / best)
    rows += [dict(hl=hl, method=k, risk_ratio=np.mean(v)) for k, v in
acc.items()]
t2 = pd.DataFrame(rows).pivot(index="method", columns="hl",
values="risk_ratio").round(3)
print("[Test 2] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân
tố:"); display(t2)
assert (t2.loc["EW-RIE*"] < t2.loc["EW-RIE (q_eff)"]).all()
```

```
[Test 2] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố:
```

hl	30	60
method		
EW-NL-LW	1.636	1.269
EW-RIE (q_eff)	1.782	1.629
EW-RIE*	1.290	1.210
EW-Sample	2.929	2.048
Oracle	1.278	1.201

▶ v3 · §1b (bổ sung). Kiểm thử cho N > T_eff và WeSpeR

Test 2b: như Test 2 của thầy nhưng thêm half-life 15 (N = 100 > T_eff ≈ 43), bản NL-LW cũ/mới và EW-WeSpeR. **Test 3:** mô hình thuận của WeSpeR (phổ quần thể → phổ mẫu). **Test 4:** đường co d(λ) khi N > T_eff.

In [11]:
```
def gmv(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def factor_corr(N, rng):
    sec = rng.integers(0, 8, N); B = np.zeros((N, 9))
    B[:, 0] = rng.uniform(.4, .8, N); B[np.arange(N), 1 + sec] =
rng.uniform(.2, .5, N)
    return renorm(B @ B.T + np.diag(rng.uniform(.3, .6, N)))

rows = []
for hl in (15, 30, 60):
    w = ew_weights(T_, hl); acc = {}
    for rep in range(6):
        Ct = factor_corr(N_, rng); X = rng.standard_normal((T_, N_)) @
np.linalg.cholesky(Ct).T
        C, _ = ew_corr(X, w); lam, V = np.linalg.eigh(C)
        Co = renorm((V * np.einsum("ij,jk,ki->i", V.T, Ct, V)) @ V.T)    #
oracle RIE
        wo = gmv(Ct); best = wo @ Ct @ wo
        for k, Ch in {"EW-Sample": C, "EW-RIE (q_eff)": rie_corr(C, N_ *
(w**2).sum()),
                      "EW-NL-LW (v2)": renorm(nl_lw_S_v2(C, 1 /
(w**2).sum())), "EW-NL-LW": renorm(nl_lw_S(C, 1 / (w**2).sum())),
                      "EW-WeSpeR": ew_wesper_corr(C, w), "EW-RIE*":
ew_rie_corr(C, w), "Oracle": Co}.items():
            wh = gmv(Ch); acc.setdefault(k, []).append(wh @ Ct @ wh / best)
    rows += [dict(hl=hl, method=k, risk_ratio=np.mean(v)) for k, v in
acc.items()]
t2b = pd.DataFrame(rows).pivot(index="method", columns="hl",
values="risk_ratio").round(3)
teff = {hl: 1 / (ew_weights(T_, hl)**2).sum() for hl in (15, 30, 60)}
print("[Test 2b] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố,
N=100, T=250; T_eff =",
      {h: round(v) for h, v in teff.items()}); display(t2b)
assert (t2b.loc["EW-RIE*", [30, 60]] < t2b.loc["EW-RIE (q_eff)", [30,
60]]).all()
assert (t2b.loc["EW-NL-LW"] <= t2b.loc["EW-NL-LW (v2)"] + 1e-9).all()
# bản sửa không kém v2
assert np.isclose(t2b.loc["EW-NL-LW", 60], t2b.loc["EW-NL-LW (v2)", 60])
# N < T_eff: trùng v2
assert (t2b.loc["EW-WeSpeR"] / t2b.loc["Oracle"] < 1.03).all()
# WeSpeR trong 3% của oracle
```

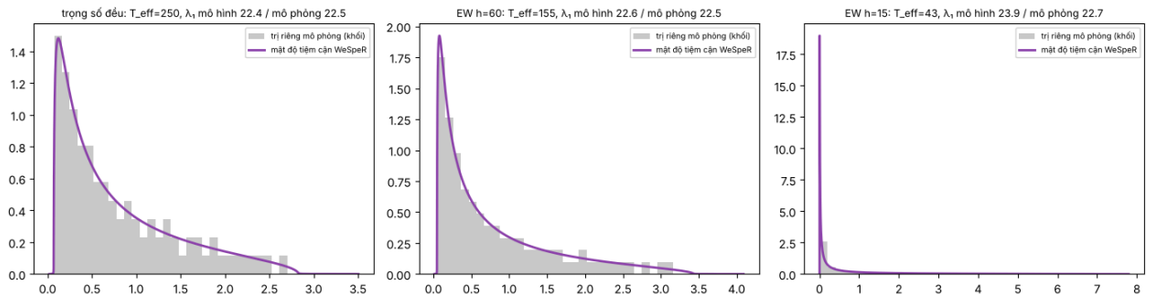
```
[Test 2b] rủi ro GMV / rủi ro tối ưu (1 = hoàn hảo), mô hình nhân tố, N=100, T=
250; T_eff = {15: 43, 30: 86, 60: 155}
```

hl	15	30	60
method			
EW-NL-LW	1.422	1.744	1.183
EW-NL-LW (v2)	12.441	1.849	1.183
EW-RIE (q_eff)	1.741	1.753	1.573
EW-RIE*	1.718	1.313	1.152
EW-Sample	5.598	2.797	1.881
EW-WeSpeR	1.386	1.317	1.153
Oracle	1.367	1.301	1.147

Nhận xét Test 2b (và Test 2 của thầy).

- **N > T_eff (h = 15, T_eff ≈ 43):** EW-NL-LW bản của thầy cho rủi ro gấp **12,4 lần** tối ưu; bản sửa còn 1,42. EW-WeSpeR 1,39 gần oracle (1,37). EW-RIE* 1,72 — kém hơn WeSpeR và EW-NL-LW (bản sửa) ở vùng q_eff > 2, nhưng vẫn tốt hơn EW-Sample (5,6) và EW-RIE (q_eff) (1,74).
- **h = 30 (T_eff ≈ 86, q_eff = 1,16):** bản sửa vẫn tốt hơn bản cũ (1,744 so với 1,849); EW-RIE* 1,313 và EW-WeSpeR 1,317 sát oracle (1,301).
- **h = 60 (N < T_eff):** hai bản NL-LW **trùng nhau** (1,183), đúng như thiết kế; EW-RIE* 1,152 ≈ WeSpeR 1,153 ≈ oracle 1,147.
- **Lưu ý Test 2 của thầy (ô ngay trên, giữ nguyên mã):** nó gọi `nl_lw_S` nên giờ dùng bản sửa. Ở h = 30 cột EW-NL-LW là 1,636 (bản gốc của thầy in 1,822) vì N > T_eff; các cột khác không đổi.

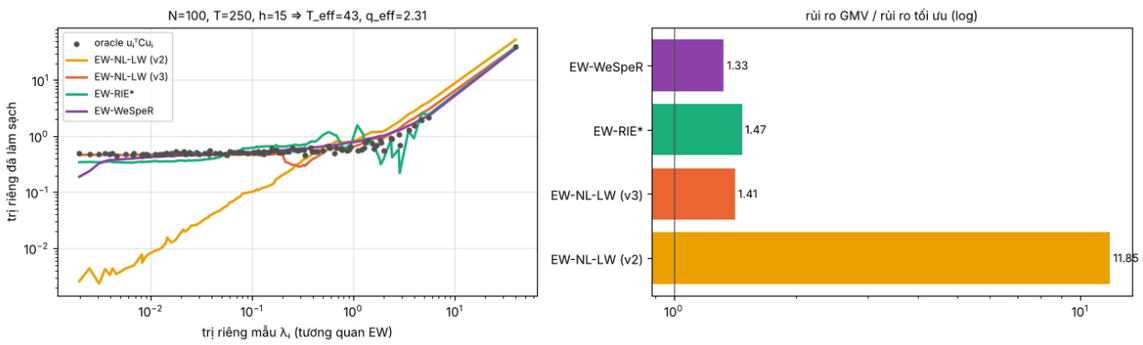
In [12]:
```
# [Test 3] Mô hình thuận WeSpeR: cho τ (1 mode thị trường + khô´i), so phân
vị mật độ tiệm cận với trị riêng mô pho̓ng (TB 20 lâ`n)
rng3 = np.random.default_rng(1); fig, ax = plt.subplots(1, 3, figsize=(14,
3.8)); rows3 = []
tau3 = np.concatenate([np.geomspace(.3, 2, N_ - 1), [25.]]); tau3 *= N_ /
tau3.sum()
for a, hl in zip(ax, (None, 60, 15)):
    w = np.full(T_, 1 / T_) if hl is None else ew_weights(T_, hl)
    ev = np.sort(np.mean([np.linalg.eigvalsh((Y * w[:, None]).T @ Y) for Y in
(rng3.standard_normal((T_, N_)) * np.sqrt(tau3) for _ in range(20))], 0))
    x = np.unique(np.concatenate([np.geomspace(ev[0] * .25, ev[-1] * 1.6,
1500), ev[-1] * np.linspace(.6, 1.4, 80)])); wv, wm = _wsp_bins(w)
    e, m, res = _wsp_cold(x, 2e-3 * x, tau3, wv, wm, N_); qh =
_wsp_quantiles(x, m, N_, N_)
    rows3.append(dict(hl="đều" if hl is None else hl, T_eff=1 / (w**2).sum(),
max_residual=res.max(),
                      median_rel_err=np.median(np.abs(qh / ev - 1)),
lam1_model=qh[-1], lam1_sim=ev[-1]))
    lab = "trọng sô´đều" if hl is None else f"EW h={hl}"; bulk = x < ev[-2] *
1.3
    a.hist(ev[:-1], bins=30, density=True, color="#c9c9c9", label="trị riêng
mô phỏng (khôí)")
    a.plot(x[bulk], m.imag[bulk] / np.pi * N_ / (N_ - 1), color="#8e44ad",
lw=2, label="mật độ tiệm cận WeSpeR")
    a.set_title(f"{lab}: T_eff={1 / (w**2).sum():.0f}, λ₁ mô hình
{qh[-1]:.1f} / mô phỏng {ev[-1]:.1f}", fontsize=9); a.legend(fontsize=7)
plt.tight_layout(); plt.show(); t3 = pd.DataFrame(rows3);
display(t3.round(4))
assert (t3.max_residual < 1e-9).all() and (t3.median_rel_err < 0.05).all()
```



	hl	T_eff	max_residual	median_rel_err	lam1_model	lam1_sim
0	đều	250.0000	0.0	0.0121	22.3715	22.4828
1	60	154.8626	0.0	0.0134	22.5773	22.4951
2	15	43.2877	0.0	0.0170	23.9092	22.7177

Nhận xét Test 3 (mô hình thuận WeSpeR). Với cùng phổ quần thể τ (99 trị riêng khối + 1 mode thị trường $\tau_1 = 25$), mật độ tiệm cận do phương trình tự đồng nhất sinh ra (đường tím) trùng với histogram trị riêng mô phỏng ở cả trọng số đều, EW h = 60 và EW h = 15. Sai số tương đối trung vị của phân vị chỉ 1,2–1,7%, phần dư Newton ≈ 0 (máy tính), λ_1 mô hình 22,4–23,9 so với mô phỏng 22,5–22,7. Sai lệch lớn nhất ở h = 15 (T_eff ≈ 43): ở đây N hữu hạn và mode thị trường bị "loang" mạnh nhất. Kết luận: bước "thuận" của EW-WeSpeR (từ τ ra phổ mẫu) đúng, nên phép khớp ngược τ ở các mục sau có cơ sở.

In [13]:
```
# [Test 4] Đường co d(λ) khi N > T_eff (h = 15, T_eff ≈ 43): NL-LW v2 vs v3,
EW-RIE*, EW-WeSpeR, oracle uᵢᵀ C uᵢ
rng4 = np.random.default_rng(4); w = ew_weights(T_, 15); Ct = factor_corr(N_,
rng4)
X = rng4.standard_normal((T_, N_)) @ np.linalg.cholesky(Ct).T; C, _ =
ew_corr(X, w); lam, V = np.linalg.eigh(C)
orc = np.einsum("ij,jk,ki->i", V.T, Ct, V)
dd = {"EW-NL-LW (v2)": renorm(nl_lw_S_v2(C, 1 / (w**2).sum())), "EW-NL-LW
(v3)": renorm(nl_lw_S(C, 1 / (w**2).sum())),
      "EW-RIE*": ew_rie_corr(C, w), "EW-WeSpeR": ew_wesper_corr(C, w)}
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].scatter(lam, orc, s=12, color="#52514e", label="oracle uᵢᵀCuᵢ",
zorder=3)
for (k, Ch), col in zip(dd.items(), ["#eda100", "#eb6834", "#1baf7a",
"#8e44ad"]):
    ax[0].plot(lam, np.einsum("ij,jk,ki->i", V.T, Ch, V), lw=2, color=col,
label=k)
ax[0].set_xscale("log"); ax[0].set_yscale("log"); ax[0].set_xlabel("trị riêng
mẫu λᵢ (tương quan EW)"); ax[0].set_ylabel("trị riêng đã làm sạch")
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3); ax[0].set_title(f"N={N_}, T=
{T_}, h=15 ⇒ T_eff={1 / (w**2).sum():.0f}, q_eff={N_ * (w**2).sum():.2f}",
fontsize=10)
r4 = {k: (lambda wh: wh @ Ct @ wh / (gmv(Ct) @ Ct @ gmv(Ct)))(gmv(Ch)) for k,
Ch in dd.items()}
ax[1].barh(list(r4), list(r4.values()), color=["#eda100", "#eb6834",
"#1baf7a", "#8e44ad"]); ax[1].axvline(1, color="#52514e", lw=.8)
for i, v in enumerate(r4.values()): ax[1].text(v, i, f" {v:.2f}",
va="center", fontsize=9)
ax[1].set_xscale("log"); ax[1].set_title("rủi ro GMV / rủi ro tối ưu (log)",
fontsize=10); plt.tight_layout(); plt.show()
# Kiểm tra thêm: với c <= 1 hai bản trùng nhau tuyệt đối; với mẫu không
trọng số N > T (est_nllw) bản v3 hữu hạn, PSD
Xs = rng4.standard_normal((250, 100)); S_ = np.cov(Xs, rowvar=False); assert
np.allclose(nl_lw_S(S_, 249), nl_lw_S_v2(S_, 249))
Xw = rng4.standard_normal((80, 100)); ev_ = np.linalg.eigvalsh(est_nllw(Xw));
assert np.isfinite(ev_).all() and ev_[0] > 0
print("[Test 4] c <= 1: v3 ≡ v2 ✔ | N=100 > T=80: est_nllw hữu hạn, xác định
dương ✔ | tỷ số rủi ro:", {k: round(v, 3) for k, v in r4.items()})
```




```
[Test 4] c <= 1: v3 ≡ v2 ✔ | N=100 > T=80: est_nllw hữu hạn, xác định dương ✔ |
tỷ số rủi ro: {'EW-NL-LW (v2)': np.float64(11.852), 'EW-NL-LW (v3)': np.float64
(1.414), 'EW-RIE*': np.float64(1.471), 'EW-WeSpeR': np.float64(1.327)}
```

Nhận xét Test 4 (N > T_eff). Khi h = 15 (T_eff ≈ 43 < N = 100, q_eff = 2,31), bản v2 của EW-NL-LW áp công thức p ≤ n cho c > 1. Hình trái cho thấy nó kéo các trị riêng nhỏ xuống tới 10^{-2}–10^{-3}, trong khi oracle $u_i^T C u_i$ nằm phẳng quanh 0,5. GMV dồn tỷ trọng vào các hướng "giả rủi ro thấp" này nên rủi ro gấp **11,9 lần** tối ưu. Bản v3 (nhánh p > n) gán chung một giá trị $\tilde{d}_0$ cho không gian gần-null và bám oracle: tỷ số rủi ro còn 1,41. EW-WeSpeR tốt nhất (1,33). EW-RIE* (1,47) bắt đầu dao động ở vùng λ ≈ 1–5 (đường xanh răng cưa), dấu hiệu công thức làm trơn η = N^{-1/2} kém ổn định khi q_eff > 2. Test 2b (bảng trên) cho cùng kết luận. Ở h = 15: EW-NL-LW bản thầy 12,44 → bản sửa 1,42; WeSpeR 1,39 ≈ oracle 1,37; EW-RIE* 1,72. Ở h = 60 (N < T_eff) hai bản NL-LW trùng nhau (1,183).

2. Dữ liệu

Ba lựa chọn qua `DATA_SOURCE`. Kết quả chuẩn: `P` (giá đóng cửa điều chỉnh, dòng = ngày, cột = mã) và `V` (giá trị giao dịch, cùng kích thước).

▶ v3 · T1.0 — lấy dữ liệu `data_hose/`

Thư mục dữ liệu có sẵn cạnh notebook thì dùng; nếu không (chạy trên web) thì tải 5 file từ `raw.githubusercontent.com/chinguyenkhac/data_HOSE/main/data_hose` (repo public). Hàm nạp của thầy (ô kế tiếp, **không sửa**) đọc `prices.csv`, `values.csv` ở thư mục hiện hành nên ô này đổi thư mục sang `DATA_DIR` và ô sau khi nạp sẽ đổi lại.

In [14]:
```
import urllib.request
RAW_URL =
"https://raw.githubusercontent.com/chinguyenkhac/data_HOSE/main/data_hose"
HOSE_FILES = ["prices.csv", "values.csv", "volumes.csv",
"security_master.csv", "missing_tickers.csv"]
if DATA_DIR is None or not all(os.path.exists(f"{DATA_DIR}/{f}") for f in
HOSE_FILES):
    DATA_DIR = os.path.abspath(f"{WORKDIR}/data_hose"); os.makedirs(DATA_DIR,
exist_ok=True)
    for f in HOSE_FILES:
        if not os.path.exists(f"{DATA_DIR}/{f}"):
urllib.request.urlretrieve(f"{RAW_URL}/{f}", f"{DATA_DIR}/{f}")
    print("Đã tải từ GitHub →", DATA_DIR)
else:
    print("Dùng dữ liệu có sẵn:", DATA_DIR)
_CWD0 = os.getcwd(); os.chdir(DATA_DIR)      # để hàm nạp của thầy đọc
"prices.csv", "values.csv"
```

```
Dùng dữ liệu có sẵn: /home/user/data_HOSE/data_hose
```

In [15]:
```
def sh(cmd, cwd=None):
    r = subprocess.run(cmd, shell=True, cwd=cwd, capture_output=True,
text=True)
    if r.returncode: print(r.stderr[-500:])
    return r.returncode

def load_github(workdir):
    # Giá ngày 2006–03/2020
(github.com/88d52bdba0366127fffca9dfa93895/vnstock-data);
    # danh sách mã HOSE (github.com/vuthanhdatt/web-indicator, năm 2022 → có
survivorship bias)
    px_repo, ls_repo = f"{workdir}/vnstock-data", f"{workdir}/web-indicator"
    if not os.path.isdir(f"{px_repo}/tickers"):
        sh(f"git clone -q --depth 1 --filter=blob:none --sparse
https://github.com/88d52bdba0366127fffca9dfa93895/vnstock-data {px_repo}")
        sh("git sparse-checkout set tickers", cwd=px_repo)
    if not os.path.isdir(f"{ls_repo}/datas/hose"):
        sh(f"git clone -q --depth 1 --filter=blob:none --sparse
https://github.com/vuthanhdatt/web-indicator {ls_repo}")
        sh("git sparse-checkout set datas/hose", cwd=ls_repo)
    hose = {f[:-4].lower() for f in os.listdir(f"{ls_repo}/datas/hose")}
    px, val = {}, {}
    for f in sorted(os.listdir(f"{px_repo}/tickers")):
        t = f[:-4]
        if t not in hose: continue
        d = pd.read_csv(f"{px_repo}/tickers/{f}", parse_dates=
["date"]).drop_duplicates("date").set_index("date")
        d = d[d["close"] > 0]; px[t.upper()] = d["close"]; val[t.upper()] =
d["close"] * d["volume"]
    return pd.DataFrame(px).sort_index(), pd.DataFrame(val)

def load_csv(price_path, value_path=None):
    P = pd.read_csv(price_path, index_col=0, parse_dates=True).sort_index()
    V = pd.read_csv(value_path, index_col=0,
parse_dates=True).reindex(P.index) if value_path else P * 0 + 1
    return P, V

def load_vnstock(symbols, start="2006-01-01", end="2025-12-31"):
    # T1: API vnstock thay đổi theo phiên bản — KIỂM TRA lại trước khi
dùng (pip install -U vnstock).
    from vnstock import Vnstock
    px, val = {}, {}
    for s in symbols:
        try:
            d = Vnstock().stock(symbol=s,
source="VCI").quote.history(start=start, end=end, interval="1D")
            d = d.set_index(pd.to_datetime(d["time"]))
            px[s] = d["close"]; val[s] = d["close"] * d["volume"]   # LƯU Ý:
kiểm tra giá đã điều chỉnh chưa
        except Exception as e:
            print(s, "lỗi:", e)
        time.sleep(0.3)
    return pd.DataFrame(px).sort_index(), pd.DataFrame(val).sort_index()

if DATA_SOURCE == "github":
    P, V = load_github(WORKDIR)
elif DATA_SOURCE == "csv":
    P, V = load_csv("prices.csv", "values.csv")      # T1: thay đường dẫn
else:
    P, V = load_vnstock(["VNM", "FPT", "VCB"])         # T1: thay bằng danh
sách đầy đủ, gồm mã đã huỷ niêm yết

P = P[P.notna().sum(1) >= 20]; V = V.reindex(P.index)
print(f"{P.shape[1]} mã, {P.shape[0]} phiên: {P.index[0].date()} →
{P.index[-1].date()}")
```

```
493 mã, 2957 phiên: 2015-01-05 → 2026-11-10
```

▶ v3 · T1.1 — chỉ giữ quan sát trong khoảng niêm yết HOSE

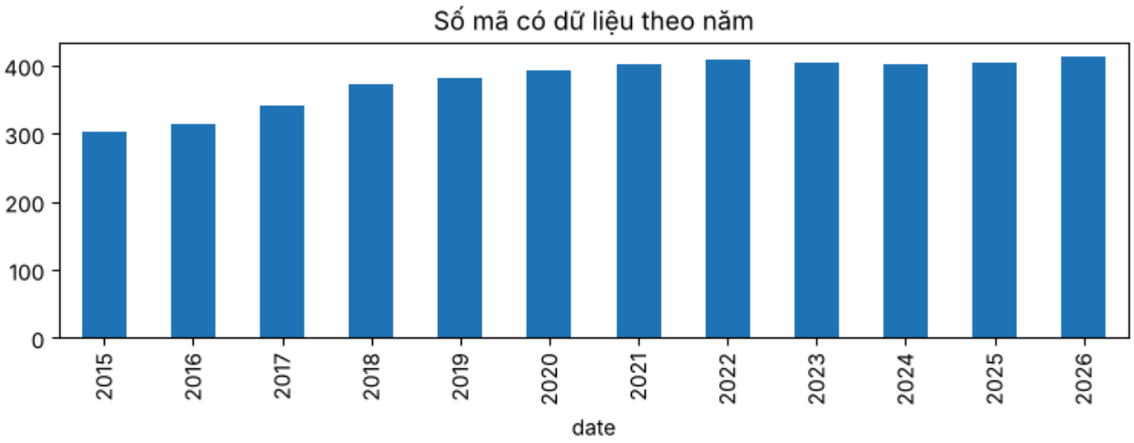
`security_master.csv`: mỗi dòng = một giai đoạn niêm yết trên HOSE (ngày niêm yết/hủy, lý do, sàn trước/sau). Quan sát giá nằm ngoài khoảng này (ví dụ mã còn giao dịch sau khi chuyển sàn) bị loại. Mã hủy niêm yết **được giữ** trong khoảng thời gian còn niêm yết — không survivorship bias.

In [16]:
```
os.chdir(_CWD0)
P_RAW = P.copy()
Q = pd.read_csv(f"{DATA_DIR}/volumes.csv", index_col=0,
parse_dates=True).reindex(index=P_RAW.index, columns=P_RAW.columns)
MASTER_RAW = pd.read_csv(f"{DATA_DIR}/security_master.csv", parse_dates=
["listing_date", "delisting_date"])
MISSING = pd.read_csv(f"{DATA_DIR}/missing_tickers.csv")
MASTER = MASTER_RAW.drop_duplicates(subset=["ticker", "listing_date",
"delisting_date"]).reset_index(drop=True)
INWIN = pd.DataFrame(False, index=P_RAW.index, columns=P_RAW.columns)      #
mặt nạ khoảng niêm yết (một mã có thể có nhiều giai đoạn)
for r in MASTER.itertuples():
    if r.ticker in INWIN:
        INWIN.loc[(INWIN.index >= r.listing_date) & ((INWIN.index <=
r.delisting_date) if pd.notna(r.delisting_date) else True), r.ticker] = True
OUTSIDE = P_RAW.notna() & ~INWIN
P = P_RAW.where(INWIN); V = V.reindex(index=P_RAW.index,
columns=P_RAW.columns).where(INWIN); Q = Q.where(INWIN)
P = P[P.notna().sum(1) >= 20]; V = V.reindex(P.index); Q = Q.reindex(P.index)
DELISTED = set(MASTER.loc[MASTER.delisting_date.notna(), "ticker"]) -
set(MASTER.loc[MASTER.delisting_date.isna(), "ticker"])
LAST_DELIST = MASTER.dropna(subset=
["delisting_date"]).groupby("ticker").delisting_date.max()
print(f"{P.shape[1]} mã có giá, {P.shape[0]} phiên: {P.index[0].date()} →
{P.index[-1].date()} | security_master: {len(MASTER_RAW)} dòng "
      f"({len(MASTER_RAW) - len(MASTER)} dòng trùng bị bỏ),
{MASTER.ticker.nunique()} mã, {len(DELISTED)} mã đã rời HOSE | "
      f"thiếu giá: {len(MISSING)} mã | quan sát ngoài khoảng niêm yết (đã
loại): {int(OUTSIDE.sum().sum())}")
```

```
493 mã có giá, 2957 phiên: 2015-01-05 → 2026-11-10 | security_master: 513 dòng
(0 dòng trùng bị bỏ), 512 mã, 104 mã đã rời HOSE | thiếu giá: 19 mã | quan sát
ngoài khoảng niêm yết (đã loại): 101
```

In [17]:
```
# Kiểm tra chất lượng dữ liệu
R_all = np.log(P).diff()
print("Tỷ lệ |r| > ngưỡng:", f"{(R_all.abs() >
CONFIG['MAX_ABS_RET']).sum().sum() / R_all.notna().sum().sum():.4%}")
print("Tỷ lệ phiên giá không đổi (r = 0):", f"{(R_all == 0).sum().sum() /
R_all.notna().sum().sum():.1%}")
P.notna().groupby(P.index.year).any().sum(axis=1).plot.bar(figsize=(9, 2.5),
title="Số mã có dữ liệu theo năm"); plt.show()
```

```
Tỷ lệ |r| > ngưỡng: 0.0005%
Tỷ lệ phiên giá không đổi (r = 0): 15.3%
```

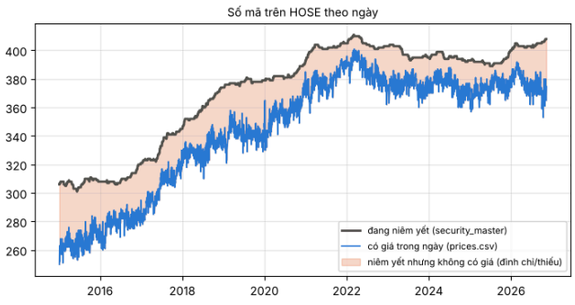


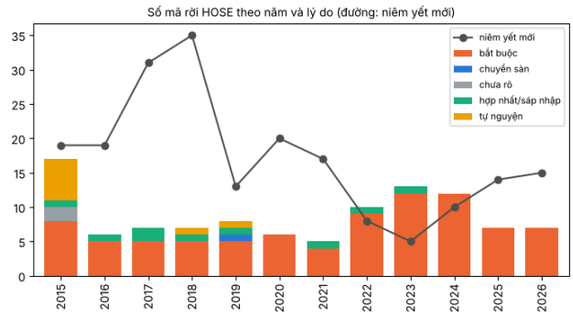
▶ v3 · T1a — Lịch sử niêm yết và mã hủy niêm yết

In [18]:
```
days = P.index; listed = pd.Series(0, index=days)
for r in MASTER.itertuples():
    listed[(days >= r.listing_date) & ((days <= r.delisting_date) if
pd.notna(r.delisting_date) else True)] += 1
withpx = P.notna().sum(1)
RN = {"bat_buoc": "bắt buộc", "hop_nhat": "hợp nhất/sáp nhập", "tu_nguyen":
"tự nguyện", "chuyen_san": "chuyển sàn", "chua_ro": "chưa rõ"}
dl = MASTER.dropna(subset=["delisting_date"]); dl = dl[dl.delisting_date >=
P.index[0]]
DY = dl.assign(year=dl.delisting_date.dt.year,
reason=dl.reason.map(RN).fillna("chưa rõ")).pivot_table(index="year",
columns="reason", values="ticker", aggfunc="count", fill_value=0)
NL = MASTER[MASTER.listing_date >=
P.index[0]].listing_date.dt.year.value_counts().sort_index()
pxed = set(P.columns)
T1A = pd.DataFrame({"Niêm yết mới": NL, "Hủy niêm yết": DY.sum(1),
                    "Hủy – có giá":
dl[dl.ticker.isin(pxed)].delisting_date.dt.year.value_counts(),
                    "Hủy – thiếu giá":
dl[~dl.ticker.isin(pxed)].delisting_date.dt.year.value_counts()}).fillna(0).a
stype(int)
T1A.loc["Tổng"] = T1A.sum(); display(T1A.T)
print("Lý do hủy niêm yết (2015–2026):", dl.reason.map(RN).fillna("chưa
rõ").value_counts().to_dict(),
      "| sàn đến sau khi rời HOSE:",
dl.exchange_after.value_counts().to_dict())
fig, ax = plt.subplots(1, 2, figsize=(14, 4))
ax[0].plot(listed, color="#52514e", lw=2, label="đang niêm yết
(security_master)")
ax[0].plot(withpx, color="#2a78d6", lw=1.2, label="có giá trong ngày
(prices.csv)")
ax[0].fill_between(days, withpx, listed, color="#eb6834", alpha=.25,
label="niêm yết nhưng không có giá (đình chỉ/thiếu)")
ax[0].set_title("Số mã trên HOSE theo ngày", fontsize=10);
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
DY.plot.bar(stacked=True, ax=ax[1], color=["#eb6834", "#2a78d6", "#9aa0a6",
"#1baf7a", "#eda100"][:DY.shape[1]], width=.75)
ax[1].plot(range(len(DY)), NL.reindex(DY.index).fillna(0).values,
color="#52514e", marker="o", lw=1.5, label="niêm yết mới")
ax[1].set_title("Số mã rời HOSE theo năm và lý do (đường: niêm yết mới)",
fontsize=10); ax[1].legend(fontsize=8); ax[1].set_xlabel("")
plt.tight_layout(); plt.show()
```

	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	Tổng
Niêm yết mới	19	19	31	35	13	20	17	8	5	10	14	15	206
Hủy niêm yết	17	6	7	7	8	6	5	10	13	12	7	7	105
Hủy – có giá	11	3	4	5	8	5	4	7	13	12	7	7	86
Hủy – thiếu giá	6	3	3	2	0	1	1	3	0	0	0	0	19

```
Lý do hủy niêm yết (2015–2026): {'bắt buộc': 85, 'hợp nhất/sáp nhập': 9, 'tự ng
uyện': 8, 'chưa rõ': 2, 'chuyển sàn': 1} | sàn đến sau khi rời HOSE: {'UPCOM':
81, 'OTC(VCI-now)': 11, 'OTC/UNKNOWN': 11, 'UPCOM(VCI-now)': 2}
```





Nhận xét T1a.

- **Số mã niêm yết** trên HOSE tăng từ ≈ 305 (2015) lên ≈ 410 (2022), sau đó đi ngang 390–410. Số mã có giá mỗi ngày luôn thấp hơn 25–50 mã (vùng cam). Phần chênh này gồm mã bị đình chỉ/không khớp lệnh trong ngày và 19 mã thiếu giá điều chỉnh từ VCI. Do đó dữ liệu **không được** nội suy, và quy tắc "≥ 95% phiên có dữ liệu" khi chọn universe là cần thiết.
- **105 mã rời HOSE** trong 2015–2026, đều đặn 5–17 mã/năm, không năm nào bằng 0. Đa số là **hủy bắt buộc** (85 mã, chủ yếu chuyển xuống UPCoM: 81). 2015 cao bất thường (17 mã, nhiều mã hủy tự nguyện), 2022–2024 tăng lại (10–13 mã/năm). Đây đúng là nhóm mã mà dữ liệu chỉ gồm mã đang niêm yết sẽ bỏ sót ⇒ nguồn survivorship bias (đo ở T1c).
- **19/105 mã đã hủy không lấy được giá** (VCI không còn mã), tập trung ở 2015–2018 và 2022. Survivorship bias vì thế **chưa được loại hoàn toàn**: phần còn sót là mã hủy sớm, thường nhỏ và kém thanh khoản, ít khả năng vào top-100 GTGD.

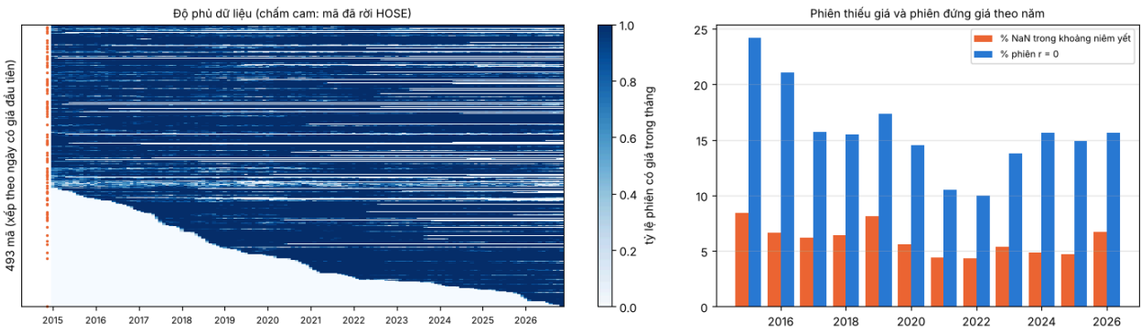
▶ v3 · T1b — Kiểm tra chất lượng và đánh giá tổng quan bộ dữ liệu

In [19]:

```
R_all = np.log(P).diff(); r_s = np.expm1(R_all); nobs =
int(R_all.notna().sum().sum())
inw = INWIN.reindex(P.index)
QA = pd.Series({
    "Số phiên": len(P), "Số mã có giá": P.shape[1], "Số mã trong
security_master": MASTER.ticker.nunique(),
    "Mã thiếu giá (missing_tickers)": len(MISSING), "Mã thiếu giá đã hủy niêm
yết": int(MISSING.delisting_date.notna().sum()),
    "Dòng trùng trong security_master": len(MASTER_RAW) - len(MASTER),
    "Quan sát ngoài khoảng niêm yết (đã loại)": int(OUTSIDE.sum().sum()),
    "% ô NaN trong khoảng niêm yết": 100 * (inw & P.isna()).sum().sum() /
inw.sum().sum(),
    "Số quan sát lợi suất": nobs, "% phiên r = 0": 100 * (R_all ==
0).sum().sum() / nobs,
    "% |r| > 7,5% (ngoài biên độ ±7%)": 100 * (r_s.abs() > .075).sum().sum()
/ nobs,
    "% |r| > 15% (coi là lỗi, đặt = 0)": 100 * (R_all.abs() >
CONFIG["MAX_ABS_RET"]).sum().sum() / nobs,
    "% quan sát giá điều chỉnh < 2.000đ": 100 * (P < 2000).sum().sum() /
P.notna().sum().sum(),
    "Phiên khối lượng = 0 mà có giá": int(((Q == 0) &
P.notna()).sum().sum()),
    "Giá ≤ 0 hoặc NaN/inf trong giá": int((P <= 0).sum().sum() +
np.isinf(P).sum().sum()),
})
display(QA.to_frame("giá trị").style.format(precision=3, decimal=",",
thousands=" "))
print("Mã có quan sát ngoài khoảng niêm yết:", OUTSIDE.sum()[OUTSIDE.sum() >
0].sort_values(ascending=False).head(8).to_dict())
# Hình 1: độ phủ dữ liệu theo tháng (mã xếp theo ngày có giá đầu tiên) +
tỷ lệ NaN theo năm
cov_m = P.notna().resample("ME").mean().T; first = P.apply(lambda s:
s.first_valid_index()); cov_m = cov_m.loc[first.sort_values().index]
fig, ax = plt.subplots(1, 2, figsize=(14, 4.2),
gridspec_kw=dict(width_ratios=[1.6, 1]))
im = ax[0].imshow(cov_m.values, aspect="auto", cmap="Blues",
interpolation="nearest", vmin=0, vmax=1)
yr = cov_m.columns.year; tk = [i for i in range(len(yr)) if i == 0 or yr[i]
!= yr[i - 1]]
ax[0].set_xticks(tk, yr[tk], fontsize=8); ax[0].set_ylabel(f"{P.shape[1]} mã
(xếp theo ngày có giá đầu tiên)"); ax[0].set_yticks([])
dlm = [i for i, t in enumerate(cov_m.index) if t in DELISTED];
ax[0].scatter(np.full(len(dlm), -1.5), dlm, s=2, color="#eb6834",
clip_on=False)
plt.colorbar(im, ax=ax[0], label="tỷ lệ phiên có giá trong tháng");
ax[0].set_title("Độ phủ dữ liệu (chấm cam: mã đã rời HOSE)", fontsize=10)
nan_y = (inw & P.isna()).groupby(P.index.year).sum().sum(1) /
inw.groupby(P.index.year).sum().sum(1) * 100
z_y = (R_all == 0).groupby(P.index.year).sum().sum(1) /
R_all.notna().groupby(P.index.year).sum().sum(1) * 100
ax[1].bar(nan_y.index - .2, nan_y.values, .4, color="#eb6834", label="% NaN
trong khoảng niêm yết")
ax[1].bar(z_y.index + .2, z_y.values, .4, color="#2a78d6", label="% phiên r =
0")
ax[1].legend(fontsize=8); ax[1].grid(axis="y", alpha=.3);
ax[1].set_title("Phiên thiếu giá và phiên đứng giá theo năm", fontsize=10)
plt.tight_layout(); plt.show()
print("% NaN theo năm:", nan_y.round(1).to_dict()); print("% r=0 theo năm:",
z_y.round(1).to_dict())
```

	giá trị
Số phiên	2 957,000
Số mã có giá	493,000
Số mã trong security_master	512,000
Mã thiếu giá (missing_tickers)	19,000
Mã thiếu giá đã hủy niêm yết	19,000
Dòng trùng trong security_master	0,000
Quan sát ngoài khoảng niêm yết (đã loại)	101,000
% ô NaN trong khoảng niêm yết	5,889
Số quan sát lợi suất	988 548,000
% phiên r = 0	15,343
% |r| > 7,5% (ngoài biên độ ±7%)	0,043
% |r| > 15% (coi là lỗi, đặt = 0)	0,001
% quan sát giá điều chỉnh < 2.000đ	1,721
Phiên khối lượng = 0 mà có giá	0,000
Giá ≤ 0 hoặc NaN/inf trong giá	0,000

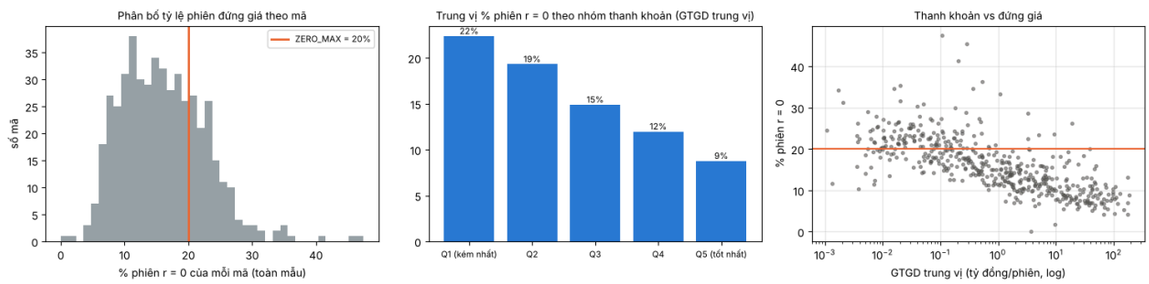
```
Mã có quan sát ngoài khoảng niêm yết: {'SAP': 101}
```




```
% NaN theo năm: {2015: 8.5, 2016: 6.6, 2017: 6.2, 2018: 6.4, 2019: 8.1, 2020:
5.6, 2021: 4.5, 2022: 4.3, 2023: 5.4, 2024: 4.9, 2025: 4.7, 2026: 6.7}
% r=0 theo năm: {2015: 24.2, 2016: 21.1, 2017: 15.7, 2018: 15.5, 2019: 17.4, 20
20: 14.5, 2021: 10.5, 2022: 10.0, 2023: 13.8, 2024: 15.7, 2025: 14.9, 2026: 15.
7}
```

Nhận xét T1b – độ phủ. Bảng trên cho thấy bộ dữ liệu "sạch" về mặt kỹ thuật: không có giá ≤ 0/NaN/inf, không có phiên khối lượng = 0 mà vẫn có giá, không có dòng trùng trong security_master. Chỉ **101 quan sát nằm ngoài khoảng niêm yết**, tất cả thuộc mã SAP, và đã được loại. Hình trái: mã mới niêm yết xuất hiện dần theo đường chéo, mã đã rời HOSE (chấm cam) phân bố khắp các năm, các vạch trắng ngang là giai đoạn đình chỉ dài. Hình phải: tỷ lệ ô thiếu trong khoảng niêm yết ổn định ở 4–9%/năm (toàn mẫu 5,9%). **Tỷ lệ phiên đứng giá (r = 0) cao hơn nhiều**: 24% năm 2015, giảm còn 10% ở giai đoạn bùng nổ thanh khoản 2021–2022, rồi quay lại ≈ 15% năm 2024–2026. r = 0 là đặc trưng của thị trường (bước giá, mã ít giao dịch), không phải lỗi dữ liệu. Nhưng nó làm ước lượng hiệp phương sai lệch về phía "rủi ro thấp giả" ⇒ lý do đưa lọc ZERO_MAX vào cấu hình chuẩn.

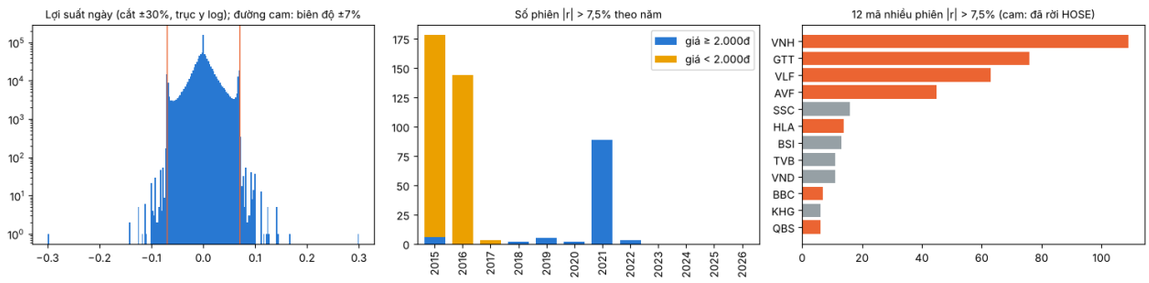
In [20]:
```
# Hình 2: phiên đứng giá (r = 0) và thanh khoản — cơ sở cho lọc ZERO_MAX
zr_t = (R_all == 0).sum() / R_all.notna().sum(); mv = V.median()
liq_q = pd.qcut(mv.rank(method="first"), 5, labels=["Q1 (kém nhất)", "Q2",
"Q3", "Q4", "Q5 (tốt nhất)"])
fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
ax[0].hist(zr_t * 100, bins=40, color="#9aa0a6"); ax[0].axvline(100 *
CONFIG["ZERO_MAX"], color="#eb6834", lw=2, label=f"ZERO_MAX =
{CONFIG['ZERO_MAX']:.0%}")
ax[0].set_xlabel("% phiên r = 0 của mỗi mã (toàn mẫu)"); ax[0].set_ylabel("số mã"); ax[0].legend(fontsize=8); ax[0].set_title("Phân bố tỷ lệ phiên đứng giá theo mã", fontsize=10)
g = zr_t.groupby(liq_q).median() * 100; ax[1].bar(g.index.astype(str),
g.values, color="#2a78d6")
for i, v in enumerate(g.values): ax[1].text(i, v, f"{v:.0f}%", ha="center",
va="bottom", fontsize=8)
ax[1].set_title("Trung vị % phiên r = 0 theo nhóm thanh khoản (GTGD trung
vị)", fontsize=10); ax[1].tick_params(axis="x", labelsize=8)
ax[2].scatter(mv / 1e9, zr_t * 100, s=8, alpha=.5, color="#52514e");
ax[2].set_xscale("log")
ax[2].axhline(100 * CONFIG["ZERO_MAX"], color="#eb6834", lw=1.5);
ax[2].set_xlabel("GTGD trung vị (tỷ đồng/phiên, log)"); ax[2].set_ylabel("%
phiên r = 0")
ax[2].set_title("Thanh khoản vs đứng giá", fontsize=10); ax[2].grid(alpha=.3)
plt.tight_layout(); plt.show()
print(f"Mã có > {CONFIG['ZERO_MAX']:.0%} phiên r=0 (toàn mẫu): {(zr_t >
CONFIG['ZERO_MAX']).sum()}/{len(zr_t)}; trung vị theo nhóm thanh khoản:",
g.round(1).to_dict(),
      f"| tương quan hạng (GTGD, %r=0) = {mv.rank().corr(zr_t.rank()):.2f}")
```




```
Mã có > 20% phiên r=0 (toàn mẫu): 135/493; trung vị theo nhóm thanh khoản: {'Q1
(kém nhất)': 22.4, 'Q2': 19.3, 'Q3': 14.9, 'Q4': 12.0, 'Q5 (tốt nhất)': 8.8} |
tương quan hạng (GTGD, %r=0) = -0.77
```

Nhận xét T1b – đứng giá và thanh khoản. Đứng giá gắn chặt với thanh khoản: tương quan hạng giữa GTGD trung vị và % phiên r = 0 là **−0,77**. Trung vị % r = 0 giảm đều từ 22,4% (nhóm kém thanh khoản nhất) xuống 8,8% (nhóm tốt nhất). Toàn mẫu có 135/493 mã có > 20% phiên r = 0 (vượt đường cam). Ngay cả nhóm Q4–Q5, thường được chọn vào universe top-100, vẫn có mã vượt ngưỡng (hình phải). Vì thế ngưỡng 20% loại được các mã "đứng giá" mà không phải chỉ là mã ít giao dịch, và vẫn giữ đủ mã cho N = 100 ở hầu hết các kỳ (hình T1b – bối cảnh).

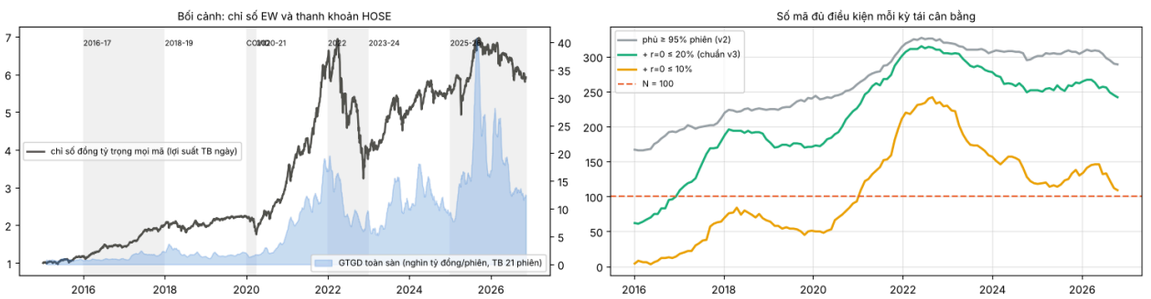
In [21]:
```
# Hình 3: phân phố´i lợi suấ´t ngày, phiên vượt biên độ theo năm (tách mã giá
< 2.000đ – nhiề˜u bước giá 100đ cu̇a VCI)
big = r_s.abs() > .075; low = P.shift(1) < 2000
by = pd.DataFrame({"giá ≥ 2.000đ": (big &
~low).groupby(P.index.year).sum().sum(1), "giá < 2.000đ": (big &
low).groupby(P.index.year).sum().sum(1)})
fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
rv = r_s.values.ravel(); rv = rv[np.isfinite(rv)]
ax[0].hist(np.clip(rv, -.3, .3), bins=241, color="#2a78d6", log=True)
for x_ in (-.07, .07): ax[0].axvline(x_, color="#eb6834", lw=1)
ax[0].set_title("Lợi suất ngày (cắt ±30%, trục y log); đường cam: biên độ
±7%", fontsize=10)
by.plot.bar(stacked=True, ax=ax[1], color=["#2a78d6", "#eda100"], width=.75);
ax[1].set_title("Sô´phiên |r| > 7,5% theo năm", fontsize=10);
ax[1].set_xlabel("")
k_ = (r_s.abs() > .075).sum().sort_values(ascending=False).head(12);
ax[2].barh(k_.index[::-1], k_.values[::-1], color=["#eb6834" if t in DELISTED
else "#9aa0a6" for t in k_.index[::-1]])
ax[2].set_title("12 mã nhiều phiên |r| > 7,5% (cam: đã rời HOSE)",
fontsize=10)
plt.tight_layout(); plt.show()
print(f"Phiên |r|>7,5%: {int(big.sum().sum())}, trong đó giá < 2.000đ:
{int((big & low).sum().sum())}; |r|>15% (đặt 0): {int((R_all.abs() >
CONFIG['MAX_ABS_RET']).sum().sum())}")
print(f"Phân vị lợi suất ngày: 0,1% = {np.quantile(rv, .001):.4f}, 99,9% =
{np.quantile(rv, .999):.4f}, độ nhọn dư = {pd.Series(rv).kurt():.1f}")
```




```
Phiên |r|>7,5%: 426, trong đó giá < 2.000đ: 319; |r|>15% (đặt 0): 5
Phân vị lợi suất ngày: 0,1% = -0.0702, 99,9% = 0.0705, độ nhọn dư = 1.6
```

Nhận xét T1b – lợi suất. Phân phối lợi suất ngày bị chặn rõ ở biên độ ±7% (phân vị 0,1%/99,9% = −7,02%/+7,05%). Chỉ có **426 phiên |r| > 7,5%** (0,043% số quan sát). Trong đó **319 phiên (75%) ở mã có giá điều chỉnh < 2.000đ**: VCI làm tròn giá tới 100đ, nên ở mức giá này một bước giá đã vượt biên độ. Đây là nhiễu bước giá, không phải lỗi. Các phiên này tập trung ở 2015–2016 (cột vàng) và ở vài mã đã rời HOSE (VNH, GTT, VLF, AVF – thanh cam). **Điểm cần rà soát:** năm 2021 có ≈ 90 phiên vượt biên độ ở mã giá ≥ 2.000đ (cột xanh), xảy ra 04–08/2021 và tập trung ở nhóm chứng khoán (SSC 16, BSI 13, TVB 11, VND 11). Nhiều khả năng đây là sai lệch của hệ số điều chỉnh giá quanh ngày GDKHQ (đợt phát hành quyền mua/cổ tức cổ phiếu năm 2021), không phải biến động thật. Cần đối chiếu với giá chưa điều chỉnh. Chỉ **5 phiên |r| > 15%** bị coi là lỗi và đặt = 0 (MAX_ABS_RET). Ảnh hưởng tới backtest là không đáng kể, vì các mã này hiếm khi lọt vào top-100 GTGD.

In [22]:
```
# Hình 4: bối cảnh thị trường + số mã đủ điều kiện mỗi kỳ tái cân bằng theo quy tắc chọn universe
R_q = np.log(P).diff(); R_q = np.expm1(R_q.mask(R_q.abs() > CONFIG["MAX_ABS_RET"], 0.0)); Wn = CONFIG["WINDOW"]; elig = []
for s in range(Wn + 1, len(P) - 1, CONFIG["REB"]):
    win = R_q.iloc[s - Wn:s]; ok = (win.notna().mean() >= .95) & P.iloc[s - 1].notna(); zs = (win == 0).mean()
    elig.append(dict(date=P.index[s], cov95=int(ok.sum()), z20=int((ok & (zs <= .2)).sum()), z10=int((ok & (zs <= .1)).sum())))
ELIG = pd.DataFrame(elig).set_index("date")
ew_idx = (1 + R_q.mean(1, skipna=True).fillna(0)).cumprod(); val_d = V.sum(1) / 1e12
PERIODS_PLOT = [("2016-17", "2016-01-01", "2017-12-31"), ("2018-19", "2018-01-01", "2019-12-31"), ("COVID", "2020-01-01", "2020-03-31"),
                ("2020-21", "2020-04-01", "2021-12-31"), ("2022", "2022-01-01", "2022-12-31"), ("2023-24", "2023-01-01", "2024-12-31"), ("2025-26", "2025-01-01", "2026-12-31")]
fig, ax = plt.subplots(1, 2, figsize=(15, 4))
ax[0].plot(ew_idx, color="#52514e", lw=1.8, label="chỉ số đồng tỷ trọng mọi mã (lợi suất TB ngày)"); ax2 = ax[0].twinx()
ax2.fill_between(val_d.index, val_d.rolling(21).mean(), color="#2a78d6", alpha=.25, label="GTGD toàn sàn (nghìn tỷ đồng/phiên, TB 21 phiên)")
for i, (lab, a_, b_) in enumerate(PERIODS_PLOT):
    ax[0].axvspan(pd.Timestamp(a_), min(pd.Timestamp(b_), P.index[-1]), color=["#f3f3f3", "#ffffff"][i % 2], zorder=0)
    ax[0].text(pd.Timestamp(a_), ew_idx.max() * .97, lab, fontsize=7)
ax[0].set_title("Bối cảnh: chỉ số EW và thanh khoản HOSE", fontsize=10); ax[0].legend(fontsize=8, loc="center left"); ax2.legend(fontsize=8, loc="lower right")
ax[1].plot(ELIG.cov95, color="#9aa0a6", lw=2, label="phủ ≥ 95% phiên (v2)"); ax[1].plot(ELIG.z20, color="#1baf7a", lw=2, label="+ r=0 ≤ 20% (chuẩn v3)")
ax[1].plot(ELIG.z10, color="#eda100", lw=2, label="+ r=0 ≤ 10%"); ax[1].axhline(CONFIG["N"], color="#eb6834", ls="--", label=f"N = {CONFIG['N']}")
ax[1].set_title("Số mã đủ điều kiện mỗi kỳ tái cân bằng", fontsize=10); ax[1].legend(fontsize=8); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
print("Số mã đủ ĐK (min / trung vị / max):", {c: (int(ELIG[c].min()), int(ELIG[c].median()), int(ELIG[c].max())) for c in ELIG},
      f"| số kỳ < N: z20 = {(ELIG.z20 < CONFIG['N']).sum()}, z10 = {(ELIG.z10 < CONFIG['N']).sum()} / {len(ELIG)}")
```




```
Số mã đủ ĐK (min / trung vị / max): {'cov95': (166, 278, 327), 'z20': (61, 249, 315), 'z10': (3, 114, 242)} | số kỳ < N: z20 = 11, z10 = 60 / 129
```

Nhận xét T1b – bối cảnh và universe.

- **Hình trái.** Chỉ số đồng tỷ trọng toàn sàn tăng ≈ 6–7 lần, chủ yếu trong 2020–2021 (bùng nổ, GTGD toàn sàn tăng từ < 5 lên > 20 nghìn tỷ/phiên). Sau đó giảm mạnh năm 2022 và biến động lớn trong 2025–2026 (đỉnh thanh khoản > 35 nghìn tỷ/phiên). Lưu ý: chỉ số này gồm cả mã nhỏ nên tăng mạnh hơn VN-Index.
- **Hình phải** (số mã đủ điều kiện mỗi kỳ):
  - Quy tắc v2 (phủ ≥ 95%) luôn có 166–327 mã.
  - Thêm lọc r = 0 ≤ 20% (chuẩn v3) giữ trung vị 249 mã. Riêng **11/129 kỳ đầu (2016) có < 100 mã** đủ điều kiện và bị bỏ (không hạ N), nên mẫu OOS chuẩn v3 bắt đầu 12/2016 thay vì 01/2016 (118 kỳ, 2.475 phiên).
  - Ngưỡng 10% quá chặt: **60/129 kỳ** thiếu mã (chỉ 3–242 mã), nên chỉ dùng làm kiểm tra độ vững (T2b).

Đánh giá tổng quan bộ dữ liệu `data_hose` (T1).

Tiêu chí	Đánh giá	Bằng chứng
Phạm vi	01/2015 → 11/2026, 2.957 phiên, 493 mã có giá / 512 mã từng niêm yết	§2, T1a
Survivorship bias	**Đã xử lý phần lớn**: giữ 86/105 mã đã rời HOSE; còn thiếu 19 mã hủy (chủ yếu 2015–2018, mã nhỏ)	T1a, T1c
Tính toàn vẹn	Tốt: không giá âm/NaN lạ, không khối lượng 0 có giá, không dòng trùng; 101 quan sát ngoài khoảng niêm yết (mã SAP) đã loại	bảng T1b
Thiếu dữ liệu	5,9% ô trong khoảng niêm yết bị thiếu (đình chỉ/không khớp); không nội suy	T1b – độ phủ
Đứng giá	15,3% phiên r = 0, gắn với thanh khoản kém (ρ hạng = −0,77) ⇒ lọc ZERO_MAX 20%	T1b – đứng giá
Ngoại lai	0,043% phiên vượt biên độ, 75% do bước giá ở mã < 2.000đ; 5 phiên > 15% đặt = 0	T1b – lợi suất
Hạn chế	GTGD = giá *điều chỉnh* × khối lượng (xấp xỉ, chỉ dùng để xếp hạng thanh khoản); giá VCI làm tròn 100đ; nghi vấn hệ số điều chỉnh ở ≈ 90 phiên 2021 (nhóm chứng khoán); lợi suất ngày hủy niêm yết = 0 (không phạt mất giá khi hủy bắt buộc)	README dữ liệu

Kết luận: dữ liệu đủ chất lượng cho nghiên cứu GMV với universe top-100 thanh khoản. Hai điểm cần nêu trong bài: (i) 19 mã hủy thiếu giá, (ii) giả định lợi suất 0 sau khi hủy niêm yết. Ở T1c, survivorship bias làm Vol thấp hơn ≈ 0,26–0,39 pp và lợi suất cao hơn 1,1–2,6 pp/năm, nhưng không đổi thứ hạng các phương pháp.

3. Danh mục & backtest

Vũ trụ động: mỗi kỳ chọn N mã có giá trị GD trung vị cao nhất trong cửa sổ (≥95% phiên có dữ liệu). Mã ngừng GD ngoài mẫu → lợi suất 0.

In [23]:

```
def gmv_unconstrained(S):
    w = np.linalg.solve(S, np.ones(len(S))); return w / w.sum()

def gmv_long_only(S, iters=3000):
    # FISTA + chiếu lên đơn hình (khớp SLSQP, nhanh hơn)
    n = len(S)
    def proj(v):
        u = np.sort(v)[::-1]; css = np.cumsum(u) - 1
        k = np.nonzero(u - css / np.arange(1, n + 1) > 0)[0][-1]; return
np.maximum(v - css[k] / (k + 1), 0)
    L = 2 * np.linalg.eigvalsh(S)[-1]; w = y = np.full(n, 1 / n); t = 1.0
    for _ in range(iters):
        wn = proj(y - 2 * S @ y / L); tn = (1 + np.sqrt(1 + 4*t*t)) / 2
        y = wn + (t - 1) / tn * (wn - w)
        if np.abs(wn - w).max() < 1e-9: return wn
        w, t = wn, tn
    return w

PERIODS = [("2008-09 Khủng hoảng TC", "2008-01-01", "2009-12-31"), ("2010-12
Lạm phát/nợ xấú", "2010-01-01", "2012-12-31"),
           ("2013-17 Ổn định", "2013-01-01", "2017-12-31"), ("2018-19 Điều
chỉnh", "2018-01-01", "2019-12-31"),
           ("2020Q1 COVID", "2020-01-01", "2020-03-31"), ("2020-21 Bùng nổ",
"2020-04-01", "2021-12-31"),
           ("2022 Sụt giảm", "2022-01-01", "2022-12-31"), ("2023-25", "2023-
01-01", "2025-12-31")]

def run_backtest(P, V, estimators, cfg, long_only):
    N, W, REB = cfg["N"], cfg["WINDOW"], cfg["REB"]
    R = np.log(P).diff(); R = np.expm1(R.mask(R.abs() > cfg["MAX_ABS_RET"],
0.0))
    dates = R.index; names = list(estimators)
    rets = {k: [] for k in names}; idx = []; reb_dates = []
    Wp = {k: pd.Series(dtype=float) for k in names}
    st = {k: dict(turn=[], neff=[], wmax=[], short=[]) for k in names}
    for s in range(W + 1, len(dates) - 1, REB):
        win = R.iloc[s - W:s]; ok = (win.notna().mean() >= .95) & P.iloc[s -
1].notna()
        if ok.sum() < N: continue
        tick = V.iloc[s - W:s].loc[:,
ok].median().sort_values(ascending=False).index[:N]
        X = win[tick].fillna(0).values; Y = R.iloc[s:s + REB]
[tick].fillna(0).values
        idx.extend(dates[s:s + REB]); reb_dates.append(dates[s])
        for k in names:
            S = estimators[k](X)
            w = gmv_long_only(S) if long_only else gmv_unconstrained(S)
            ws = pd.Series(w, index=tick)
            if len(Wp[k]):
                u = ws.index.union(Wp[k].index)
                st[k]["turn"].append((ws.reindex(u).fillna(0) -
Wp[k].reindex(u).fillna(0)).abs().sum())
            Wp[k] = ws; rets[k].extend(Y @ w)
            st[k]["neff"].append(1 / (w**2).sum()); st[k]
["wmax"].append(w.max()); st[k]["short"].append(np.clip(-w, 0, None).sum())
    out = {}
    for k in names:
        r = pd.Series(rets[k], index=idx); rn = r.copy()
        rn.loc[reb_dates[1:]] -= cfg["COST"] * np.array(st[k]["turn"])
        out[k] = dict(r=r, r_net=rn, m={
            "Vol %": r.std() * np.sqrt(252) * 100, "LN/năm %": r.mean() * 252
* 100,
            "Sharpe": r.mean() / r.std() * np.sqrt(252), "Sharpe sau phí":
rn.mean() / rn.std() * np.sqrt(252),
            "Turnover/kỳ": np.mean(st[k]["turn"]), "Số mã hiệu dụng":
np.mean(st[k]["neff"]),
            "Tỷ trọng max %": np.mean(st[k]["wmax"]) * 100, "Short %":
np.mean(st[k]["short"]) * 100})
    return out

def block_bootstrap_ci(ra, rb, block=21, B=2000, seed=0):
    # CI 95% cho Vol(a) − Vol(b), %/năm. (T3: bổ sung kiểm định Ledoit–Wolf
```

```
2011)
    rng = np.random.default_rng(seed); n = len(ra); nb = n // block; d =
np.empty(B)
    for i in range(B):
        ix = (rng.integers(0, n - block, nb)[:, None] +
np.arange(block)).ravel()
        d[i] = (ra[ix].std() - rb[ix].std()) * np.sqrt(252) * 100
    return np.percentile(d, [2.5, 97.5])

def summarize(out, pairs):
    tab = pd.DataFrame({k: o["m"] for k, o in out.items()}).T
    per = pd.DataFrame({k: {p: o["r"][s:e].std() * np.sqrt(252) * 100 for p,
s, e in PERIODS if len(o["r"][s:e]) > 20}
                        for k, o in out.items()}).T
    ci = pd.DataFrame({f"{a} vs {b}": block_bootstrap_ci(out[a]["r"].values,
out[b]["r"].values)
                       for a, b in pairs if a in out and b in out}, index=
["CI 2.5%", "CI 97.5%"]).T
    ci["Có ý nghĩa"] = (ci["CI 97.5%"] < 0) | (ci["CI 2.5%"] > 0)
    return tab, per, ci
```

▶ v3 · §3b. Lọc mã đứng giá (`ZERO_MAX`) — không sửa `run_backtest` của thầy

Bản `run_backtest` dưới đây được **sinh từ chính mã nguồn của thầy** (`inspect.getsource`), chỉ chèn thêm 1 dòng lọc ngay trước kiểm tra `ok.sum() < N` : `ok &= (win == 0).mean() <= cfg.get("ZERO_MAX", 1.0)` — chỉ dùng cửa sổ quá khứ. Nếu `ZERO_MAX = 1.0` (hoặc không có trong cfg) thì kết quả **trùng hệt** bản của thầy (assert bên dưới). Kỳ không đủ N mã sau lọc bị bỏ (không hạ N). Ngoài ra ghi log universe mỗi kỳ (`ULOG`).

In [24]:
```
import inspect
ULOG = []
_src = inspect.getsource(run_backtest)
_chk = "        if ok.sum() < N: continue\n"
_sel = "        tick = V.iloc[s - W:s].loc[:,
ok].median().sort_values(ascending=False).index[:N]\n"
assert _src.count(_chk) == 1 and _src.count(_sel) == 1, "mã nguồn
run_backtest của thầy đã đổi"
exec(_src.replace("def run_backtest(", "def run_backtest_x(")
         .replace(_chk, '        ok &= (win == 0).mean() <=
cfg.get("ZERO_MAX", 1.0)          # v3: lọc mã đứng giá (cửa sổ quá khứ)\n' +
_chk)
         .replace(_sel, _sel + "        ULOG.append(dict(date=dates[s],
n_eligible=int(ok.sum()), tickers=list(tick)))\n"))
run_backtest_thay = run_backtest                          # bản gốc của
thầy (đổi chiếu)
def run_backtest(*a, **k):                                # các ô của thầy
gọi run_backtest → dùng bản có lọc
    ULOG.clear(); return run_backtest_x(*a, **k)

# kiểm tra: ZERO_MAX = 1.0 ⇒ trùng hệt bản của thầy
_p, _v = P.iloc[-900:], V.iloc[-900:]
_a = run_backtest_thay(_p, _v, {"NL-LW": est_nllw}, CONFIG, True)["NL-LW"]
["r"]
_b = run_backtest(_p, _v, {"NL-LW": est_nllw}, dict(CONFIG, ZERO_MAX=1.0),
True)["NL-LW"]["r"]
assert _a.index.equals(_b.index) and np.array_equal(_a.values, _b.values)
print(f"run_backtest (có lọc) ≡ run_backtest của thầy khi ZERO_MAX = 1.0 ✔ |
cấu hình chuẩn: ZERO_MAX = {CONFIG['ZERO_MAX']:.0%}")
```

```
run_backtest (có lọc) ≡ run_backtest của thầy khi ZERO_MAX = 1.0 ✔ | cấu hình c
huẩn: ZERO_MAX = 20%
```

4. Chạy chính

Thời gian: chế độ đầy đủ trên Colab khoảng 15–30 phút (10 bộ ước lượng × 2 chế độ). Đặt `QUICK=True` để chạy thử nhanh.

In [25]:
```
ESTIMATORS = build_estimators(CONFIG)
PAIRS = [("EW-RIE*", "EW-RIE (q_eff)"), ("EW-RIE*", "EW-NL-LW"), ("EW-RIE*",
"NL-LW"), ("EW-RIE*", "RIE"),
         ("NL-LW", "Ledoit-Wolf"), ("VS-NL-LW", "NL-LW")]
RESULTS = {}
for long_only in (False, True):
    mode = "long_only" if long_only else "short_ok"; t0 = time.time()
    out = run_backtest(P, V, ESTIMATORS, CONFIG, long_only)
    RESULTS[mode] = dict(out=out, **dict(zip(["table", "periods", "ci"],
summarize(out, PAIRS))))
    print(f"\n===== {mode} | {time.time() - t0:.0f}s =====")
    display(RESULTS[mode]["table"].round(2))
    display(RESULTS[mode]["periods"].round(2))
    display(RESULTS[mode]["ci"].round(3))
print("Newton thất bại:", _STATS)
```

```
===== short_ok | 254s =====
```

	Vol %	LN/năm %	Sharpe	Sharpe sau phí	Turnover/kỳ	Số mã hiệu dụng	Tỷ trọng max %	Short %
Sample	14.58	2.52	0.17	-0.20	1.81	4.60	24.22	122.25
Ledoit-Wolf	13.63	2.55	0.19	-0.08	1.20	8.41	17.87	86.54
NL-LW	13.10	1.76	0.13	-0.05	0.81	11.52	13.98	60.78
MP-clip	13.48	1.92	0.14	-0.05	0.86	7.46	22.86	64.80
RIE	13.30	4.69	0.35	0.23	0.55	12.88	17.25	30.58
VS-NL-LW	12.49	3.15	0.25	-0.08	1.41	7.65	25.73	54.28
VS-RIE	12.63	4.15	0.33	0.09	0.99	11.22	21.34	29.08
EW-NL-LW	12.72	1.30	0.10	-0.13	1.00	8.04	22.77	58.70
EW-RIE (q_eff)	14.30	4.75	0.33	0.23	0.51	20.25	14.14	12.53
EW-WeSpeR	12.57	1.73	0.14	-0.08	0.92	8.67	21.82	53.59
EW-RIE*	12.56	1.01	0.08	-0.14	0.94	8.53	22.16	54.96

	2013-17 Ổn định	2018-19 Điều chỉnh	2020Q1 COVID	2020-21 Bùng nổ	2022 Sụt giảm	2023-25
Sample	11.11	11.85	16.85	15.15	18.09	15.01
Ledoit-Wolf	9.54	11.24	20.32	13.77	17.15	13.82
NL-LW	9.38	10.87	20.48	12.81	16.95	13.15
MP-clip	9.31	11.02	16.80	13.16	17.57	13.87
RIE	9.04	10.60	17.79	13.15	17.89	13.63
VS-NL-LW	8.33	10.11	14.67	10.65	17.49	13.18
VS-RIE	8.15	10.21	15.21	10.90	18.17	13.24
EW-NL-LW	8.78	10.42	16.65	11.60	17.25	13.26
EW-RIE (q_eff)	8.49	10.79	18.46	13.55	20.68	14.90
EW-WeSpeR	8.50	10.08	16.47	11.52	17.13	13.16
EW-RIE*	8.59	10.14	16.45	11.48	17.12	13.11

	CI 2.5%	CI 97.5%	Có ý nghĩa
EW-RIE* vs EW-RIE (q_eff)	-2.305	-1.121	True
EW-RIE* vs EW-NL-LW	-0.237	-0.094	True
EW-RIE* vs NL-LW	-0.990	-0.058	True
EW-RIE* vs RIE	-1.196	-0.208	True
NL-LW vs Ledoit-Wolf	-0.741	-0.319	True
VS-NL-LW vs NL-LW	-1.277	0.061	False

```
===== long_only | 89s =====
```

	Vol %	LN/năm %	Sharpe	Sharpe sau phí	Turnover/kỳ	Số mã hiệu dụng	Tỷ trọng max %	Short %
Sample	13.52	1.65	0.12	0.03	0.41	9.24	25.08	0.0
Ledoit-Wolf	13.64	2.36	0.17	0.09	0.38	12.63	20.06	0.0
NL-LW	13.72	2.08	0.15	0.08	0.34	14.40	16.80	0.0
MP-clip	13.67	3.36	0.25	0.16	0.41	9.09	25.93	0.0
RIE	14.02	4.44	0.32	0.24	0.35	15.32	17.52	0.0
VS-NL-LW	12.93	4.24	0.33	0.16	0.74	8.06	30.73	0.0
VS-RIE	13.30	4.73	0.36	0.20	0.68	12.83	21.99	0.0
EW-NL-LW	13.12	1.20	0.09	-0.02	0.49	9.17	26.22	0.0
EW-RIE (q_eff)	14.76	4.99	0.34	0.26	0.41	22.24	14.07	0.0
EW-WeSpeR	13.05	1.40	0.11	-0.00	0.47	9.39	25.74	0.0
EW-RIE*	13.07	1.31	0.10	-0.01	0.48	9.46	25.65	0.0

	2013-17 Ổn định	2018-19 Điều chỉnh	2020Q1 COVID	2020-21 Bùng nổ	2022 Sụt giảm	2023-25
Sample	9.61	12.06	17.02	13.21	17.58	13.86
Ledoit-Wolf	9.49	12.09	20.58	13.13	17.67	13.74
NL-LW	9.52	12.10	20.67	13.08	18.11	13.76
MP-clip	9.58	11.87	16.69	13.26	18.16	14.16
RIE	9.29	11.62	18.04	13.71	19.31	14.28
VS-NL-LW	8.66	11.27	14.13	10.76	18.78	13.64
VS-RIE	8.48	11.16	15.12	11.26	19.67	13.89
EW-NL-LW	8.84	11.45	16.14	11.79	17.77	13.68
EW-RIE (q_eff)	8.67	11.41	18.67	13.95	21.54	15.30
EW-WeSpeR	8.77	11.30	16.13	11.66	17.77	13.62
EW-RIE*	8.81	11.33	16.11	11.73	17.75	13.63

	CI 2.5%	CI 97.5%	Có ý nghĩa
EW-RIE* vs EW-RIE (q_eff)	-2.162	-1.220	True
EW-RIE* vs EW-NL-LW	-0.091	-0.025	True
EW-RIE* vs NL-LW	-1.030	-0.281	True
EW-RIE* vs RIE	-1.311	-0.564	True
NL-LW vs Ledoit-Wolf	-0.048	0.219	False
VS-NL-LW vs NL-LW	-1.481	-0.119	True

```
Newton thất bại: {'fail': 0, 'total': 52598}
```

▶ v3 · §4b. So sánh bổ sung EW-WeSpeR (khoảng tin cậy bootstrap khối)

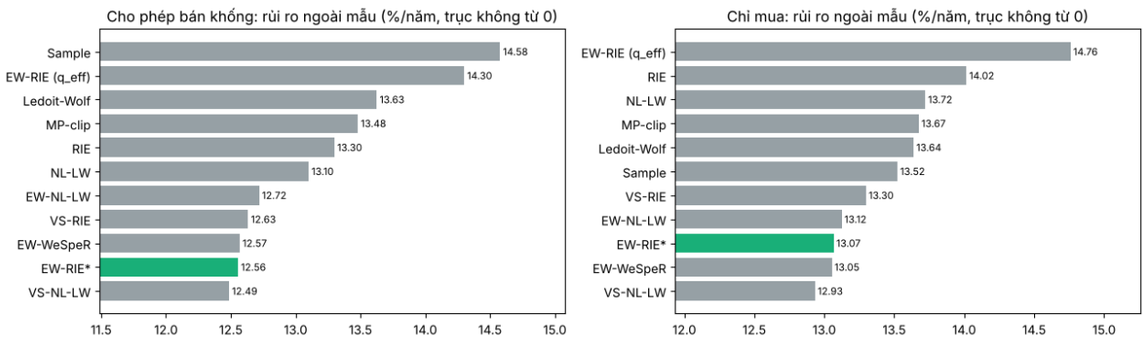
Bảng CI của ô §4 (của thầy) không có cặp với WeSpeR; ô này bổ sung.

In [26]:
```
U_FULL = list(ULOG)       # universe mỗi kỳ của lần chạy chính (chỉ mua) — dùng ở T1c
EXTRA = [("EW-RIE*", "EW-WeSpeR"), ("EW-WeSpeR", "EW-NL-LW"), ("EW-WeSpeR", "NL-LW")]
rows = []
for mode in RESULTS:
    for a, b in EXTRA:
        lo_, hi_ = block_bootstrap_ci(RESULTS[mode]["out"][a]["r"].values, RESULTS[mode]["out"][b]["r"].values)
        rows.append({"Chế độ": mode, "Cặp (a vs b)": f"{a} vs {b}", "ΔVol a−b (pp)": RESULTS[mode]["table"].loc[a, "Vol %"] - RESULTS[mode]["table"].loc[b, "Vol %"],
                     "CI 2.5%": lo_, "CI 97.5%": hi_, "Có ý nghĩa": (hi_ < 0) or (lo_ > 0)})
display(pd.DataFrame(rows).set_index(["Chế độ", "Cặp (a vs b)"]).round(3))
print("WeSpeR: dự phòng sang EW-RIE* =", _WSP_SAFE, f"| RMSE log-phân vị khớp TB = {np.mean(_WSP['cost']):.3f}")
print(f"OOS: {len(U_FULL)} kỳ ({U_FULL[0]['date'].date()} → {RESULTS['long_only']['out']['EW-RIE*']['r'].index[-1].date()}); mã đủ ĐK mỗi kỳ {min(u['n_eligible'] for u in U_FULL)}–{max(u['n_eligible'] for u in U_FULL)}")
```

		ΔVol a−b (pp)	CI 2.5%	CI 97.5%	Có ý nghĩa
Chế độ	**Cặp (a vs b)**				
short_ok	**EW-RIE* vs EW-WeSpeR**	-0.012	-0.090	0.073	False
	EW-WeSpeR vs EW-NL-LW	-0.150	-0.265	-0.039	True
	EW-WeSpeR vs NL-LW	-0.532	-0.963	-0.040	True
long_only	**EW-RIE* vs EW-WeSpeR**	0.013	-0.021	0.050	False
	EW-WeSpeR vs EW-NL-LW	-0.070	-0.113	-0.027	True
	EW-WeSpeR vs NL-LW	-0.668	-1.041	-0.295	True

```
WeSpeR: dự phòng sang EW-RIE* = {'fallback': 0, 'total': 118} | RMSE log-phân vị khớp TB = 0.037
OOS: 118 kỳ (2016-12-08 → 2026-11-10); mã đủ ĐK mỗi kỳ 100–315
```

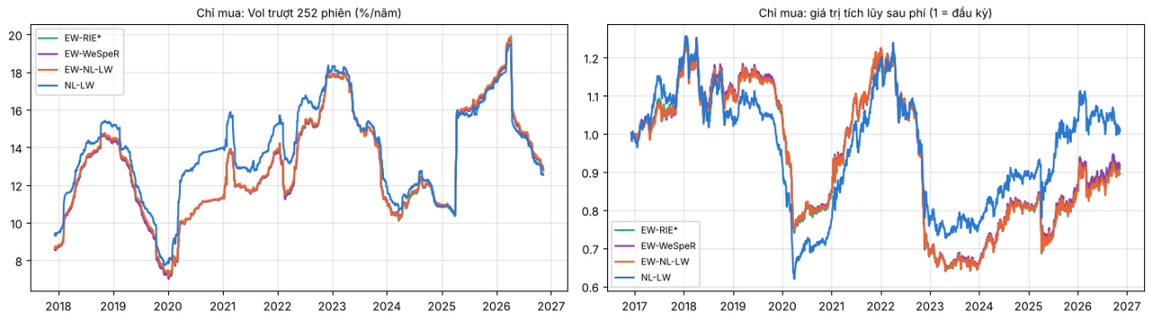
In [27]:
```
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
for a, mode, ttl in [(ax[0], "short_ok", "Cho phép bán khống"), (ax[1],
"long_only", "Chỉ mua")]:
    v = RESULTS[mode]["table"]["Vol %"].sort_values()
    a.barh(v.index, v.values, color=["#1baf7a" if k == "EW-RIE*" else
"#9aa0a6" for k in v.index])
    a.set_xlim(v.min() - 1, v.max() + 0.5); a.set_title(f"{ttl}: rủi ro ngoài
mẫu (%/năm, trục không từ 0)")
    for i, x in enumerate(v.values): a.text(x, i, f" {x:.2f}", va="center",
fontsize=8)
plt.tight_layout(); plt.show()
```



Nhận xét §4 – kết quả chính (12/2016–11/2026, 118 kỳ, ZERO_MAX 20%).

- **Bán khống:**
  - Thấp nhất là **VS-NL-LW 12,49%**, rồi đến nhóm EW: **EW-RIE* 12,56%**, **EW-WeSpeR 12,57%**, VS-RIE 12,63%, EW-NL-LW 12,72%.
  - Các đối chứng không trọng số kém rõ: NL-LW 13,10%, RIE 13,30%, Ledoit-Wolf 13,63%, Sample 14,58%. EW-RIE (q_eff) 14,30% là bản EW tệ nhất.
- **Chỉ mua:**
  - **VS-NL-LW 12,93%** thấp nhất, rồi EW-WeSpeR 13,05%, EW-RIE* 13,07%, EW-NL-LW 13,12%.
  - NL-LW 13,72%; RIE 14,02% còn cao hơn ma trận mẫu (13,52%).
- **Ý nghĩa thống kê** (block bootstrap b = 21, cả hai chế độ):
  - EW-RIE* thấp hơn có ý nghĩa so với EW-RIE (q_eff), EW-NL-LW, NL-LW và RIE.
  - **EW-RIE* và EW-WeSpeR không khác nhau** (CI chứa 0, chênh ±0,01 pp). Đây là điều lý thuyết dự đoán: cả hai ước lượng cùng một đại lượng oracle $u_i^T C u_i$ cho hiệp phương sai có trọng số, chỉ khác cách tính (trực tiếp/RIE vs khôi phục phổ/QuEST).
- **So với v2** (không lọc đứng giá): mức Vol cao hơn ≈ 1,4 pp (mẫu thời gian cũng khác) vì universe đã loại các mã "giả ít rủi ro" (T2b). **Thứ hạng thay đổi**: v2 có EW-RIE* gần như dẫn đầu, v3 thì VS-NL-LW dẫn ở cả hai chế độ (VS vs NL-LW có ý nghĩa ở chỉ mua, chưa ở bán khống).
- **Kinh tế:** GMV có lợi suất thấp (1–5%/năm) và Sharpe sau phí ≈ 0. Mục tiêu của phương pháp là rủi ro, không phải lợi suất.
- **Bảng theo giai đoạn** dùng `PERIODS` của thầy (ô §3, giữ nguyên). Nhãn "2013-17 Ổn định" thực chất chỉ gồm 12/2016–2017 (OOS bắt đầu 12/2016) và "2023-25" không gồm 2026 (dữ liệu đến 11/2026). Cần chỉnh nhãn/khoảng khi tinh chỉnh.

In [28]:
```
HL = {"EW-RIE*": "#1baf7a", "EW-WeSpeR": "#8e44ad", "EW-NL-LW": "#eb6834",
"NL-LW": "#2a78d6"}
# Rủi ro trượt 252 phiên và giá trị tích lũy (chỉ mua) cho 4 phương pháp
chính
fig, ax = plt.subplots(1, 2, figsize=(14, 4))
for k, c in HL.items():
    r = RESULTS["long_only"]["out"][k]["r"]
    ax[0].plot(r.rolling(252).std() * np.sqrt(252) * 100, color=c, lw=1.6,
label=k)
    ax[1].plot((1 + RESULTS["long_only"]["out"][k]["r_net"]).cumprod(),
color=c, lw=1.6, label=k)
ax[0].set_title("Chỉ mua: Vol trượt 252 phiên (%/năm)", fontsize=10);
ax[1].set_title("Chỉ mua: giá trị tích lũy sau phí (1 = đầu kỳ)",
fontsize=10)
for a in ax: a.grid(alpha=.3); a.legend(fontsize=8)
plt.tight_layout(); plt.show()
rv_ = pd.DataFrame({k: RESULTS["long_only"]["out"][k]["r"].rolling(252).std()
for k in HL}).dropna()
print("Tỷ lệ cửa sổ 252 phiên EW-RIE* có Vol thấp nhất trong 4 phương pháp:",
f"{(rv_.idxmin(axis=1) == 'EW-RIE*').mean():.1%}",
      "| EW-WeSpeR:", f"{(rv_.idxmin(axis=1) == 'EW-WeSpeR').mean():.1%}")
```




```
Tỷ lệ cửa sổ 252 phiên EW-RIE* có Vol thấp nhất trong 4 phương pháp: 25.0% | EW
-WeSpeR: 47.4%
```

Nhận xét §4 – theo thời gian (chỉ mua).

- **Hình trái.** Ba bộ EW (EW-RIE*, EW-WeSpeR, EW-NL-LW) gần như trùng nhau. Chúng thấp hơn NL-LW rõ nhất sau các cú sốc: COVID 2020, quý I/2022, 2025. Ở đó NL-LW (cửa sổ phẳng 250 phiên) chậm thích nghi và Vol trượt cao hơn 1–3 pp suốt nhiều tháng. Khi thị trường ổn định (2023–2024, 2025) các đường sát nhau và NL-LW có lúc thấp hơn.
- **Xét theo cửa sổ 252 phiên**, trong 4 phương pháp: EW-WeSpeR thấp nhất ở 47% cửa sổ, EW-RIE* ở 25%. Chênh lệch giữa hai bộ RMT có trọng số là rất nhỏ.
- **Hình phải.** Giá trị tích lũy sau phí của cả 4 danh mục GMV ≈ 0,9–1,0 sau 10 năm, trong khi chỉ số EW toàn sàn tăng mạnh (T1b). Danh mục rủi ro thấp trên HOSE giai đoạn này có lợi suất thấp, nhất là mất mạnh năm 2022. NL-LW có lợi suất tích lũy cao hơn từ 2023 nhờ turnover thấp hơn. Đây là đánh đổi rủi ro ↔ chi phí đã thấy ở T2.

In [29]:
```
# Lưu kết quả (tải về hoặc chép vào Drive)
with pd.ExcelWriter(f"
{WORKDIR}/results_N{CONFIG['N']}_T{CONFIG['WINDOW']}_h{CONFIG['HL_EW']}.xlsx"
) as xw:
    for mode, d in RESULTS.items():
        d["table"].round(4).to_excel(xw, sheet_name=f"{mode}_tong_hop")
        d["periods"].round(3).to_excel(xw, sheet_name=f"{mode}_giai_doan")
        d["ci"].round(4).to_excel(xw, sheet_name=f"{mode}_CI")
    pd.Series(CONFIG).to_frame("giá trị").to_excel(xw, sheet_name="CONFIG")
print("Đã lưu vào", WORKDIR)
```

```
Đã lưu vào ./rmt_work
```

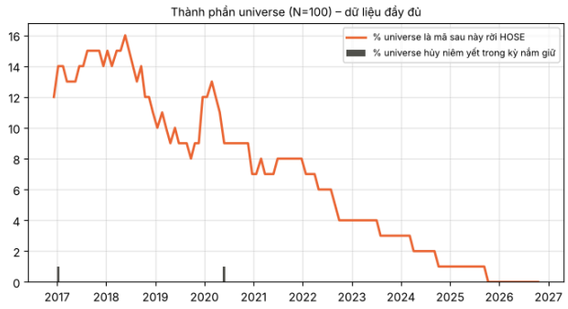
▶ v3 · T1c — Survivorship bias: dữ liệu đầy đủ vs chỉ mã còn niêm yết

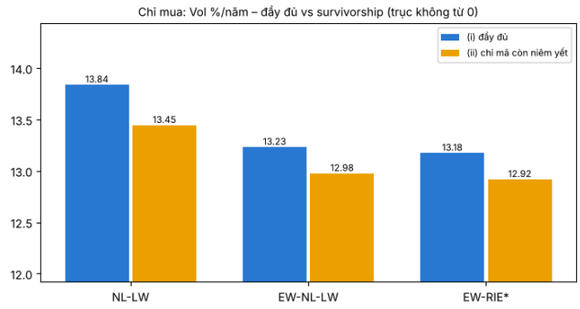
Câu hỏi: nếu chỉ dùng mã còn niêm yết đến 11/2026 (như danh sách mã hiện tại), kết quả lệch bao nhiêu? Cùng cấu hình chuẩn: (i) đầy đủ (kết quả §4), (ii) loại mọi mã đã rời HOSE. So trên ngày OOS chung. Đồng thời đo tỷ lệ vị thế bị hủy niêm yết ngay trong kỳ nắm giữ (nơi quy ước "lợi suất 0 sau ngày cuối" có tác dụng).

In [30]:
```
CORE = ["NL-LW", "EW-NL-LW", "EW-RIE*"]; MODES = {"long_only": True,
"short_ok": False}; VN = {"long_only": "Chỉ mua", "short_ok": "Bán khống"}
SURV = [c for c in P.columns if c not in DELISTED]
SURV_RES = {m: run_backtest(P[SURV], V[SURV], {k: ESTIMATORS[k] for k in
CORE}, CONFIG, lo) for m, lo in MODES.items()}
rows = []
for m in MODES:
    common = RESULTS[m]["out"]["EW-RIE*"]["r"].index.intersection(SURV_RES[m]
["EW-RIE*"]["r"].index)
    for k in CORE:
        for lab, o in (("(i) đầy đủ", RESULTS[m]["out"][k]), ("(ii) chỉ mã
còn niêm yết", SURV_RES[m][k])):
            r, rn = o["r"].loc[common], o["r_net"].loc[common]
            rows.append(dict(mode=VN[m], method=k, data=lab, **{"Vol %":
r.std() * np.sqrt(252) * 100, "LN/năm %": r.mean() * 252 * 100,
                                "Sharpe sau phí": rn.mean() / rn.std() *
np.sqrt(252), "Số ngày chung": len(common)}))
T1C = pd.DataFrame(rows)
tb = T1C.pivot_table(index=["mode", "method"], columns="data", values=["Vol
%", "LN/năm %", "Sharpe sau phí"])
for c in ("Vol %", "LN/năm %", "Sharpe sau phí"): tb[(c, "Δ (ii) − (i)")] =
tb[(c, "(ii) chỉ mã còn niêm yết")] - tb[(c, "(i) đầy đủ")]
print(f"Bảng T1c — {T1C['Số ngày chung'].iloc[0]} ngày OOS chung (chỉ mua);
{len(DELISTED)} mã đã rời HOSE bị loại ở (ii)")
display(tb.sort_index(axis=1).round(3))
sh = pd.DataFrame([dict(date=u["date"], later=np.mean([t in DELISTED for t in
u["tickers"]]),
                        within=np.mean([t in LAST_DELIST.index and u["date"]
<= LAST_DELIST[t] <= u["date"] + pd.Timedelta(days=31) for t in
u["tickers"]]))
                   for u in U_FULL]).set_index("date")
fig, ax = plt.subplots(1, 2, figsize=(14, 3.9))
ax[0].plot(sh.later * 100, color="#eb6834", lw=2, label="% universe là mã sau
này rời HOSE")
ax[0].bar(sh.index, sh.within * 100, width=20, color="#52514e", label="%
universe hủy niêm yết trong kỳ năm giữ")
ax[0].set_title("Thành phần universe (N=100) — dữ liệu đầy đủ", fontsize=10);
ax[0].legend(fontsize=8); ax[0].grid(alpha=.3)
x = np.arange(len(CORE)); L_ = T1C[T1C["mode"] == "Chỉ
mua"].pivot(index="method", columns="data", values="Vol %").reindex(CORE)
for j, (c, col) in enumerate(zip(L_.columns, ["#2a78d6", "#eda100"])):
ax[1].bar(x + (j - .5) * .38, L_[c], .36, color=col, label=c)
for j, c in enumerate(L_.columns):
    for i, v in enumerate(L_[c]): ax[1].text(i + (j - .5) * .38, v, f"
{v:.2f}", ha="center", va="bottom", fontsize=8)
ax[1].set_xticks(x, CORE); ax[1].set_ylim(L_.values.min() - 1,
L_.values.max() + .6); ax[1].legend(fontsize=8)
ax[1].set_title("Chỉ mua: Vol %/năm — đầy đủ vs survivorship (trục không từ
0)", fontsize=10)
plt.tight_layout(); plt.show()
print(f"Universe đầy đủ: TB {sh.later.mean():.1%} mã sau này rời HOSE (max
{sh.later.max():.1%}); TB {sh.within.mean():.2%} vị thế hủy niêm yết trong kỳ
năm giữ "
      f"({int(round(sh.within.mean() * 100 * len(sh)))} vị thế trong
{len(sh)} kỳ × {CONFIG['N']} mã)")
```

```
Bảng T1c — 2412 ngày OOS chung (chỉ mua); 104 mã đã rời HOSE bị loại ở (ii)
```

		LN/năm %			Sharpe sau phí			Vol %		
	data	(i) đầy đủ	(ii) chỉ mã còn niêm yết	Δ (ii) – (i)	(i) đầy đủ	(ii) chỉ mã còn niêm yết	Δ (ii) – (i)	(i) đầy đủ	(ii) chỉ mã còn niêm yết	Δ (ii) – (i)
mode	method									
Bán không	EW-NL-LW	1.031	2.339	1.308	-0.156	-0.043	0.113	12.834	12.574	-0.260
	EW-RIE*	0.718	1.809	1.092	-0.168	-0.071	0.097	12.673	12.412	-0.262
	NL-LW	1.229	3.858	2.629	-0.091	0.120	0.211	13.216	12.822	-0.394
Chỉ mua	EW-NL-LW	0.971	2.602	1.631	-0.037	0.096	0.133	13.233	12.977	-0.256
	EW-RIE*	1.001	2.606	1.606	-0.032	0.099	0.132	13.175	12.919	-0.257
	NL-LW	1.813	4.297	2.485	0.058	0.248	0.190	13.840	13.446	-0.394






```
Universe đầy đủ: TB 6.9% mã sau này rời HOSE (max 16.0%); TB 0.02% vị thế hủy niêm yết trong kỳ nắm giữ (2 vị thế trong 118 kỳ × 100 mã)
```

Nhận xét T1c – survivorship bias.

- **Hình trái.** Trong universe đầy đủ, trung bình **6,9%** (tối đa 16%, năm 2018) số mã top-100 là mã **sau này** rời HOSE. Tỷ lệ này giảm dần về 0 vào 2026 (hiển nhiên, vì chưa kịp hủy). Tuy vậy chỉ 0,02% vị thế là mã bị hủy **ngay trong kỳ nắm giữ** (2 vị thế trong 118 kỳ × 100 mã). Giả định "lợi suất 0 sau hủy" vì thế gần như không ảnh hưởng kết quả.
- **Bảng và hình phải.** Nếu chỉ dùng mã còn niêm yết (như danh sách mã hiện tại):
  - Vol **thấp hơn giả tạo 0,26 pp** (EW-RIE*, EW-NL-LW) và **0,39 pp** (NL-LW).
  - Lợi suất **cao hơn 1,1–2,6 pp/năm**, Sharpe sau phí cao hơn 0,10–0,21.

  Đây là độ lớn của survivorship bias trên HOSE: so sánh được với chính khoảng chênh giữa các bộ ước lượng (vài chục điểm cơ bản).
- **Thứ hạng giữ nguyên** (EW-RIE* ≤ EW-NL-LW < NL-LW ở cả hai bộ dữ liệu). So sánh *tương đối* giữa phương pháp vững với survivorship bias; mức Vol/lợi suất *tuyệt đối* thì không.

5. Độ nhạy half-life (T6 mở rộng thêm N, T)

In [31]:
```
RUN_SENS = False   # đặt True để chạy (~10–20 phút)
if RUN_SENS:
    rows = []
    for hl in (30, 60, 120):
        est = {"EW-NL-LW": make_ew(lambda C, w: renorm(nl_lw_S(C, 1 /
(w**2).sum())), hl), "EW-RIE*": make_ew(ew_rie_corr, hl)}
        for lo in (False, True):
            o = run_backtest(P, V, est, CONFIG, lo)
            l, u = block_bootstrap_ci(o["EW-RIE*"]["r"].values, o["EW-NL-LW"]
["r"].values)
            rows.append(dict(hl=hl, mode="long_only" if lo else "short_ok",
vol_EWNLLW=o["EW-NL-LW"]["m"]["Vol %"],
                             vol_EWRIE=o["EW-RIE*"]["m"]["Vol %"], CI_lo=l,
CI_hi=u))
    display(pd.DataFrame(rows).round(3))
```

6. Việc của nhóm (khung sẵn – điền tiếp)

In [32]:
```
# ---- T2: Độ vững ----
# Gợi ý: lặp CONFIG["REB"] in (5, 21, 63); CONFIG["COST"] in (0.0015, 0.0025,
0.004);
# xử lý phiên không giao dịch: thay r=0 bằng NaN rồi dùng hiệp phương sai
theo cặp, hoặc lọc mã có >X% phiên r=0.
# for reb in (5, 21, 63):
#     cfg = dict(CONFIG, REB=reb); out = run_backtest(P, V, {"NL-LW":
est_nllw, "EW-RIE*": ESTIMATORS["EW-RIE*"]}, cfg, False)
```

In [33]:
```
# ---- T3: Đối chứng & kiểm định ----
def est_equal_weight_proxy(X):
    # 1/N: dùng ma trận đơn vị -> GMV = 1/N
    return np.eye(X.shape[1])
# TODO: mô hình nhân tố (PCA k nhân tố), DCC-GARCH (gói `arch`/`mgarch`),
# kiểm định Ledoit & Wolf (2011) "Robust performances hypothesis testing
with the variance" (HAC + bootstrap),
# kiểm định Sharpe Jobson–Korkie với hiệu chỉnh Memmel (2003).
```

In [34]:
```
# ---- T5: Vì sao VS giúp trên HOSE? ----
# Gợi ý: (a) so sánh VS chuẩn hóa biến động riêng lẻ vs. cá nhân tố
thị trường;
# (b) đo độ chính xác dự báo biến động 21 phiên của EWMA vs. cửa sổ
phẳng trên HOSE;
# (c) phân rã lợi ích VS theo giai đoạn (bằng RESULTS[mode]["periods"]).
```

Ghi chú

- **Khai báo dùng AI:** mã nguồn được xây dựng với hỗ trợ của trợ lý AI; nhóm cần rà soát, hiểu và khai báo theo quy định của tạp chí.
- **Tính mới:** công thức tổng quát cho nhiễu nhân có trong Bun–Bouchaud–Potters (2016, 2017). Cần T4 xác nhận phần cụ thể hóa cho EWMA, chọn nhánh, xử lý ngoại lai chưa được công bố.
- **Tái lập:** cố định `seed`; ghi lại phiên bản thư viện (ô đầu) vào phụ lục.

Ghi chú v3

- **Mã hủy niêm yết:** lợi suất các ngày sau ngày giao dịch cuối = 0 (giữ tiền mặt đến kỳ tái cân bằng kế tiếp), đúng quy ước của thầy; không áp mức phạt mất giá khi hủy bắt buộc. Trên dữ liệu thực chỉ rất ít vị thế bị hủy ngay trong kỳ nắm giữ (xem T1c).
- **19 mã đã hủy không lấy được giá điều chỉnh** từ VCI (`missing_tickers.csv`), chủ yếu hủy 2015–2018: survivorship bias chưa loại hoàn toàn.
- **EW-WeSpeR** là bản cài lại bằng numpy theo các phương trình của Oriol (2024); phiên chạy không truy cập được arXiv/GitHub của bài nên cần đối chiếu với mã gốc trước khi công bố. Khởi tạo ấm từ kỳ trước nên kết quả phụ thuộc thứ tự chạy ở mức sai số tối ưu.
- **EW-NL-LW (v3)** dùng nhánh p > n của Ledoit–Wolf (2020) khi N > T_eff với T_eff = 1/Σw² không nguyên; đây là phép mở rộng hợp lý, không phải kết quả lý thuyết của LW cho trọng số mũ.
- **Chạy trên web (Colab):** mở notebook, chạy từ trên xuống; ô T1.0 tự tải `data_hose/` từ GitHub. Thời gian ≈ 10–15 phút (EW-WeSpeR chậm nhất).